Software is the horseless-carriage and the case for the Context Engineering Lifecycle

Dr Robert Li

What is software now?

What is software once agents produce and operate it?

I mean the question literally. In November 2017 Andrej Karpathy argued that neural networks "represent the beginning of a fundamental shift in how we develop software", and drew the line in two sentences: "Software 1.0 is code we write. Software 2.0 is code written by the optimization based on an evaluation criterion" (Karpathy, 2017). In June 2025 he went a step further, giving the large language model "a new designation of Software 3.0" on the grounds that "your prompts are now programs that program the LLM. And remarkably, these prompts are written in English" (Karpathy, 2025a). By February 2026 he was describing a working day in which "you are not writing the code directly 99% of the time, you are orchestrating agents who do and acting as oversight" (Karpathy, 2026). Three statements from one practitioner, spread across a little over eight years, each further from the idea that software is something a person types.

The received view in our industry is that none of this changes what software is, only how quickly it can be made: the same lifecycle, with its slow parts compressed. I will come to who holds that view, and in what words, further on. I think it is wrong. This essay argues that software itself is being redefined, not merely sped up, and that its lifecycle has to change with it.

Before I go on, I should say where I stand, because it shapes what I look for. I am an actor-network theory practitioner by doctorate (Li, 2020), a historical materialist (Li, 2011), and now a CTO, which means I spend my working days on the question above with a budget attached. I will use each lens where it explains best, and say so when they pull against each other. After this paragraph the argument has to carry itself.

I will start with the thing itself, before its lifecycle: first what is being stripped out of software, then what is left.

For as long as most of us have built it, software has carried two things at once. One is the what: the data, rules and operations a business needs. The other is the how: the particular sequence of screens, fields, menus and confirmations through which a person was expected to get at them. The two were shipped together because they had to be, since nobody could reach the what except through somebody's how. That coupling is coming apart, and the how is the half being stripped away.

A user interface has always been an opinion: a vendor's judgement about the lowest common denominator of how its users would want to work, fixed in layout and shipped

identically to everyone. Some of those opinions were very good, and they were opinions all the same.

Natural language is a different kind of interface, because what it asks of the user is something they already have. It is built on speaking, reading and writing, skills that are learned but learned universally, so language is a human-native standard that already exists at a scale no interface designer could build. That is the force of Karpathy's "remarkably" (Karpathy, 2025a): the programs are now written in the medium their users already read and write. Gesture or eye contact might be more intuitive still, but they carry too much individually particularised meaning for a model's averaging to capture, short of training a vision model on one person. Where a picture is needed, a chart, a diagram or a model in three dimensions can be rendered alongside the text.

Strip the how out and what remains is the what, made available: data, and the capabilities to act on it, exposed through an API, a Model Context Protocol (MCP) server, an agent-to-agent protocol or direct access to the data layer. In effect it is a store and a pipeline over it, and the screen moves to the far end of the wire, rendered in whichever harness (the client in which a person meets a model; more on it below) and model the user happens to be working through.

From there it is tempting to take one more step, and I took it myself when I first wrote this argument down: users will stop caring about the vendor's interface and its brand, and will care only whether the store holds the data they want, in the structure they want, reachable the way they want. Part of that is right. The part that is wrong is where the money is. This matters for the lifecycle as well as the market: whoever owns the interface is placed to define the problem of adoption for everyone else, a point I return to when I come to how a CELC is formed.

Defaults hold. The best-known demonstration of why concerns organ donation, which I offer as an illustration, not as evidence about software. Johnson and Goldstein found effective consent to donation of 4.25% to 27.5% in European countries where citizens had to opt in, against 85.9% to 99.98% where they had to opt out (Johnson and Goldstein, 2003, p.1338). They give three reasons a default sticks. People may believe "that defaults are suggestions by the policy-maker, which imply a recommended action". Deciding takes effort, "whereas accepting the default is effortless". And the default is the status quo, the reference point against which any change is judged, and since "losses loom larger than the equivalent gains", leaving it feels like a loss (p.1338). None of the three is about organs; each applies to any configuration that somebody else has chosen for you.

In software the effect is no illustration, and it has been priced at the scale harness vendors work at. In the United States' search case against Google, the District Court for the District of Columbia accepted the plaintiffs' expert's evidence that "defaults have a powerful impact on consumer decisions" (*United States v Google LLC*, 2024, p.30). It found that "In 2021, Google paid out a total of $26.3 billion in revenue share under these contracts", the contracts that bought those placements (p.105), and concluded that "All of this makes the defaults extremely valuable" (p.232). Nobody pays $26.3 billion in a single year

for something its users are about to stop caring about. And at the remedies stage the same court declined to bar Google "from making payments or offering other consideration to distribution partners for preloading or placement" of its products (*United States v Google LLC*, 2025, p.8), so paying for the default is still allowed, and every user and operator should know it.

Defaults hold often, though, not always, and both sources say where the hold gives out. The court found that "defaults are less effective when the alternative has strong brand recognition and product quality" (*United States v Google LLC*, 2024, p.212). Johnson and Goldstein's own data show the same limit from another side. The default all but decided who registered: across their countries "nearly 60 percentage points separate the two groups" (Johnson and Goldstein, 2003, p.1339). What finally happened moved far less. Where donation was the default they estimated "a 16.3% ($P < 0.02$) increase in donation", because families object, doctors hesitate and donors fail to match recipients, whatever the register says (p.1339). A default governs what people sign up to much more than what they end up with. The limit is real, but it is no comfort to anyone who does not own the default, because often is enough to be worth a great deal to whoever sets it.

In the arrangement I described above, the default is set by the harness: the client in which a person meets the model, and which decides, before they have decided anything, which model answers, which tools are to hand and how an answer is laid out. ChatGPT and Claude Desktop are harnesses in this sense, and both are highly opinionated ones. Each is a coherent product with its own view of how a conversation with a model should go, and a meaningful number of the people who use them will take that view as it comes. So the user interface has moved, from the vendor of the data to the vendor of the harness, where it now frames every service the user reaches, and that is a material advantage to the harness vendor. Users do stop caring about the data vendor's UX, then, but only because the harness's UX has replaced it: the lowest-common-denominator opinion every vendor used to ship in its own screens is still being shipped, by far fewer people.

What the harness vendor usually does not own matters just as much. The data, for the most part, belongs to someone else. So do the intermediary capabilities that expose it: the MCP and API servers in front of a store, and the agents a harness reaches over agent-to-agent protocols. The hardware, usually, is someone else's again. The harness owns very little of the what, only the conversation.

That is an intermediary's position, and we have seen one like it before. A search engine stood between people and the sources they were looking for, ranked those sources and sent the user on to them. The default harness stands in the same place and sends nobody anywhere. It takes the question, goes to the services on the user's behalf and answers in its own voice, so the user may never see a source's own interface at all. The best word I have for that role is interlocutor: the party you are actually talking to, whoever it has been talking to on your behalf. A historical materialist will recognise the structure, and "control of the means of interaction" is a fair name for what the default harness holds.

Actor-network theory has a more exact term. Michel Callon showed how an actor that defines a problem in its own terms makes itself "an obligatory passage point" for everyone else the problem involves: the point they all have to go through to get what they want (Callon, 1986). The default harness is an obligatory passage point in that plain sense. It sits between the operator or user and the whole range of services behind it, the data and capabilities reached through MCP and API servers and the agents reached over agent-to-agent protocols, and every one of those services is met through it. I will come back to obligatory passage points later, where they bear on how context itself is formed. For now it is enough to see who is standing in one.

So the value the screen used to hold concentrates in a few harnesses, while the data and the capabilities, which is to say most of what used to be the product, stay with whoever owns them. For anyone who does not own a harness, software devolves into backend, middleware and integration: the store, the pipeline over it, and the work of making both reachable through somebody else's passage point. The user still cares whether the store holds the data they want, in the structure they want, reachable the way they want. They express that care through an interface the data vendor no longer controls.

That is my working answer to the opening question, and it is not the received view. So I owe what I promised at the start: who holds the received view, and in what words. The field already has a crowded vocabulary for this shift, and I will sort the terms by what each one names: a process, an artefact or a practice.

The process names treat the change as a new lifecycle. AWS calls its version the AI-Driven Development Life Cycle, or AI-DLC (AWS, 2025). "Agentic SDLC" circulates widely with no single author. ADLC is used for two different things, a lifecycle for building agents and a lifecycle in which agents do the building, which is reason enough not to lean on it.

The artefact names say that the thing being made has changed. Karpathy's Software 2.0 and Software 3.0, which I quoted at the start, belong here. So does the academic version, SE 3.0, which proposed "intent-first, conversation-oriented development" with "AI teammates" in October 2024, ahead of the vendor wave of 2025 (Hassan et al., 2024). So does GitHub's Spec Kit, whose launch post describes a shift from "code is the source of truth" to "intent is the source of truth" (Delimarsky, 2025).

The practice names describe what a person now does. Karpathy named "vibe coding" in February 2025, a way of working in which "you fully give in to the vibes, embrace exponentials, and forget that the code even exists" (Karpathy, 2025b), and a year later gave his "current favorite" name for what came after it: "agentic engineering" (Karpathy, 2026). Between the two came "context engineering". Tobi Lütke preferred it to prompt engineering because "It describes the core skill better: the art of providing all the context for the task to be plausibly solvable by the LLM" (Lütke, 2025), and Karpathy, within the week, called it "the delicate art and science of filling the context window with just the right information for the next step" (Karpathy, 2025c).

Beyond the names, the camps differ on how much they think has changed, and their opinions run along a spectrum.

At one end is augmentation: the SDLC as it was, with assistive tools bolted on. Its canonical evidence is the controlled experiment on GitHub Copilot in which the group given it "completed the task 55.8% faster than the control group" (Peng et al., 2023). It was one task, writing an HTTP server in JavaScript, done by 95 developers. The result is real, and it is a measurement of one phase of a process that nobody proposed to change.

Next comes compression and reordering: the same phases, run faster, merged or re-sequenced. AWS rejects augmentation in so many words, warning that "Simply retrofitting AI as an assistant not only constrains its capabilities but also reinforces outdated inefficiencies", and announces that "Traditional 'sprints' are replaced by 'bolts' – shorter, more intense work cycles" (AWS, 2025). Yet the AI-DLC keeps its phases, Inception, Construction and Operations, and its AI "implements solutions only after receiving human validation". It calls itself transformative and is, structurally, a compression. Boris Tane sits at the far edge of this position, because for him collapse is not speed-up: "The stages collapsed. They didn't get faster. They merged." What is left is a loop, "Intent. Build. Observe. Repeat." (Tane, 2026). AWS's Kiro reorders rather than removes, pairing "the flow of vibe coding" with "the clarity of specs" and running requirements, then design, then tasks (Swaminathan and Singh, 2025). The 2025 report from DORA, Google's DevOps research programme, gives the position its empirical form: "AI doesn't fix a team; it amplifies what's already there" (Harvey and DeBellis, 2025).

Then transformation: a new paradigm, in which the artefact, the unit of work or the roles change. Software 3.0 is the clearest statement of it, SE 3.0 the academic one and Spec Kit's "intent is the source of truth" the vendor one. Patrick Debois puts it as plainly as anyone: "The object moving through the pipeline isn't code. It's knowledge and intent." (Debois, 2026). Anthropic's own data points the same way, with "79% of conversations on Claude Code were identified as 'automation'", against 49% on Claude.ai, and its authors ask whether developers will "shift to mostly managing and guiding these systems, rather than writing code themselves" (Anthropic, 2025). That last is a question, not a finding, and the source says so.

Against all three stand the sceptics, and theirs is the most uncomfortable data in the field. The 2025 study by METR, a nonprofit that evaluates AI models, found experienced open-source developers 19% slower with AI tools (METR, 2025). In February 2026 METR reported a new estimate that tasks took 18% less time (CI −38% to +9%) and called the new figure "an unreliable signal" (METR, 2026); it has since labelled the 2025 result "out of date" (METR, 2025). The 2024 DORA report found that "As AI adoption increased, it was accompanied by an estimated decrease in delivery throughput by 1.5%, and an estimated reduction in delivery stability by 7.2%" (DORA, 2024): correlational survey estimates, per 25% increase in adoption. By 2025 the throughput relationship had turned positive, but "AI adoption does continue to have a negative relationship with software delivery stability", and with 90% of respondents using AI, "30% report little or no trust in the code generated by AI" (Harvey and DeBellis, 2025). GitClear, a vendor, so read it with that in mind, reports from 211 million changed lines a "4x Growth in Code Clones", with cloned lines rising "from 8.3% to 12.3%" between 2021 and 2024 and moved lines, the trace of refact-

oring, falling "from 25% of changed lines in 2021, to less than 10% in 2024" (GitClear, 2025).

A fifth reading treats all of this as one more abstraction layer, the next step after assembler, high-level languages and frameworks. Martin Fowler accepts the scale of the change, putting it at "a similar degree as the change from assembler" to high-level languages, and then refuses the comfort of the analogy by quoting Birgitta Böckeler: "we're moving *sideways* into non-determinism at the same time" (Böckeler, quoted in Fowler, 2025). Fortran kept in version control compiles to the same behaviour every time. A prompt kept in version control does not reliably reproduce it.

A sixth reading is about labour. The Stanford Digital Economy Lab finds that "employment of young workers (ages 22–25) in AI-exposed occupations now stands 19% below where it would be had it kept pace with that of their less-exposed peers", working "primarily through reduced hiring", and calls its own results "early, descriptive indicators… rather than causal estimates" (Brynjolfsson et al., 2026a). Anthropic's trial of 52 developers, "mostly junior", found a "statistically significant decrease in mastery" among those using AI assistance, with quiz scores of 50% against 67%, no significant gain in speed and the largest gap on debugging (Shen and Tamkin, 2026). The worry here is what happens to people, and to skill, when the work that trained them is done by something else. I will return to it.

The seventh reading is less a position than a pattern in the evidence: human effort is moving from doing the work into reviewing and verifying it. DORA warns that "acceleration can expose weaknesses downstream" (Harvey and DeBellis, 2025). The AI-DLC makes the human a validator, with the AI implementing "only after receiving human validation" (AWS, 2025). GitClear's clones are, among other things, more code for somebody to read.

The received view I began with lives at the first two points on that spectrum: the same lifecycle, with assistance added or its slow parts compressed. I have said I think it is wrong. But every position on the spectrum is held by serious people with dated evidence, and any claim to succeed the SDLC has to say where it stands among them.

The Context Engineering Lifecycle, the CELC, sits at the transformation end of the spectrum, and it makes two claims there that, as far as I can find (I may be wrong, or have overlooked a neighbour), none of its neighbours makes together.

The first is a claim of succession. Software itself is being redefined, from a what bundled with a how into data and capabilities rendered in somebody else's harness, and a lifecycle built to produce the old thing cannot simply be run faster to produce the new one. It has to be succeeded. The second is a claim of conservation. Fred Brooks distinguished the essence of software, the difficulties inherent in its nature, from its accidents, and held that essential complexity grows non-linearly with size (Brooks, 1987, p.11). Generating code does not make that essential complexity disappear. It is mirrored in context. That is the thesis the rest of this essay argues, and it has to be earned further on, along with the questions it leaves open: whether some complexity does in fact vanish on the way, and what stops context, once it carries the complexity, from fragmenting the way code did.

The connections that used to be written into an application, between a user and a record, a record and a process, one system and the next, are now made and mediated by agents and their actions, which are the capabilities. What the agents act on, and act through, is data, and data here means more than a database: a database is one way of organising it, and there are many others. So the thing a lifecycle has to produce and keep is no longer the code that implements the product but the context through which agents assemble it, which is what I mean by saying the unit has changed. The Internet has been through a similar transformation (Li, 2025).

By context I mean something with two aspects, and the CELC needs both. Context is the medium through which the builder, the engineer or the operator now directs computation. In that sense context continues programming: directing computation is what every programming language has ever done, and Karpathy's "your prompts are now programs that program the LLM" (Karpathy, 2025a) puts the new medium in the same line as the old ones. Context is simply the newest of them. Context is also the thing that has to be managed: it is produced, kept, corrected and superseded, and it goes wrong when nobody does those things. A lifecycle through context and a lifecycle of context are the same lifecycle, seen once from the computation it directs and once from the material it is made of.

That second aspect is why I keep the term "context engineering", with one correction. Context and the context window are different things. Context is the unit of input being managed, and it comes in many forms. The window is the gateway through which context reaches a model for inference, and "window" is the better word for that. Tobi Lütke's definition already keeps the two apart, since it never mentions the window: "the art of providing all the context for the task to be plausibly solvable by the LLM" (Lütke, 2025).

The window is also, invariably, a passage point every context must go through, and the harness, which defines what reaches it, is who makes it obligatory. Whatever form context takes, it reaches the model only through the window. So there is a second passage point behind the one I described earlier: the operator or user goes through the harness, context goes through the window, and the harness, at the front of that chain, decides what reaches the window.

Karpathy's definition names the window, because it describes the moment of filling it: context engineering is "the delicate art and science of filling the context window with just the right information for the next step" (Karpathy, 2025c). In the same post he lists what an LLM application has to do "On top of context engineering itself": "break up problems just right into control flows", "pack the context windows just right", "dispatch calls to LLMs of the right kind and capability", "handle generation-verification UIUX flows", and "a lot more - guardrails, security, evals, parallelism, prefetching", which is why, for him, "context engineering is just one small piece of an emerging thick layer of non-trivial software that coordinates individual LLM calls (and a lot more) into full LLM apps". I do not read that against him. Once context and the window are kept apart, his thick layer describes the same thing I do, and anyone who has built a data platform will recognise its shape. The control flows are ETL pipelines, with context extracted, transformed and

loaded. Those pipelines have parameters that need adjusting, which is to say configuration, and that is where the rest of his list lives. Packing the windows is indexing. Dispatching calls to the right model is data routing. The generation-verification flows are the display layer an analyst works from, the dashboard. Guardrails and evals are the data rule engines. Hold on to that shape; it is how I answer the most serious objection below.

The CELC is only a label for the lifecycle of that work. It is not a new coinage, and nothing about it is proprietary. The phrase is already in use. The earliest use I have found is a post on the OneReach.ai blog, published on 8 September 2025, which captions a figure "The Context Engineering Lifecycle" and says that "The context engineering lifecycle consists of seven critical stages": "Ingest", "Normalize", "Index", "Retrieve", "Compose", "Evaluate" and "Persist" (Slesarenko, 2025). It is not an authoritative source, and I do not cite it as one; the term is nascent, and nothing written about it is yet. Read in order, though, those seven stages describe a pipeline for context, which sits comfortably with the ETL analogy above.

The nearest neighbour is Patrick Debois's Context Development Lifecycle, the CDLC, and a reader is entitled to ask whether the CELC is the CDLC under another name. Debois starts where I do: "if context is the new bottleneck, then we need a development lifecycle built around it". He treats context "as an engineering artifact (something we generate, evaluate, distribute, and observe)", from which "a lifecycle emerges" (Debois, 2026). His ambition is large: he expects the CDLC to "reshape how we think about software development as profoundly as DevOps reshaped how we think about delivery and operations" (Debois, 2026). His account of context that goes stale and contradicts itself is close to the one I give later.

The line between us runs through both aspects of context, and through the claim. Take the object first. The CDLC's context is an artefact developed alongside code and in its service. It "needs the same engineering rigour we eventually gave to code: version control, testing, security, distribution, monitoring, and continuous improvement", and the post closes on a question: "You've invested in getting agents to write code. What have you invested in giving them the context to write it well?" (Debois, 2026). That is context as something with a lifecycle, which is one of the two aspects I mean. The other is context as the medium through which computation is now directed, and what it directs is more than agents writing code. It directs the software itself: the data and capabilities left over, as I argued at the start, once the how is stripped out. Then the claim. DevOps, the analogy Debois chooses, extended the SDLC: it joined two halves of one lifecycle, delivery and operations, and the lifecycle it joined carried on. A lifecycle modelled on DevOps extends the SDLC in the same way. The CELC claims more than an extension, and so has more to prove: the SDLC organised the work of people putting code down, that work is passing to agents, and so the lifecycle is succeeded, not supplemented. This is my answer to the first objection, that the CELC is only the CDLC renamed: it would land if the CELC were only a lifecycle of context.

The second objection is that all of this is one more abstraction layer, and in one respect I have just conceded it. If context is the newest medium for directing computation, it

stands in a line with assembler, high-level languages and frameworks. What the objection misses is the direction of the step. Böckeler's point, as Fowler quotes it, is that "we're moving *sideways* into non-determinism at the same time" (Böckeler, quoted in Fowler, 2025). The abstraction layers the objection has in mind form a ladder, each rung reducing reliably to the one below, and on every rung people still put the code down themselves: in assembler, then in a high-level language, then inside a framework. The SDLC was never about the code itself: it organised the resources and processes that let code be put down, and then used, by people and teams whose throughput is limited. So it carried on, one rung up, each time. On this rung we are no longer the ones putting the code down. The code still exists, but the SDLC ceases to be relevant to it, because the work it organised has passed to agents. The step is also taken sideways, into non-determinism, while the thing being directed is itself changing, from a bundled what and how into data and capabilities. So I concede the layer and not the conclusion the objection draws from it. The CELC is the SDLC abstracted: the lifecycle one layer up, which organises the context through which the code, and everything else, is now produced.

The third objection is that the productivity is illusory, and the sceptics' data I set out above are its case. I read the same data as complexity left unmanaged in its new location. GitClear's rising clones and falling moved lines (GitClear, 2025) are what generated code looks like when nobody refactors it: the structure that used to be maintained in code is now maintained nowhere. DORA's finding that AI adoption "does continue to have a negative relationship with software delivery stability" (Harvey and DeBellis, 2025) is the same cost arriving downstream. And METR's 2025 study, whose 19% slowdown METR has since labelled "out of date" in favour of an estimate it calls "an unreliable signal" (METR, 2025; METR, 2026), reported something more telling than its headline figure: "developers expected AI to speed them up by 24%, and even after experiencing the slowdown, they still believed AI had sped them up by 20%" (METR, 2025). A gap that wide between how the work felt and how it measured is one thing you would expect if part of the work had moved somewhere the developers were not looking; another, which I come to, is a belief that does not need the numbers. None of this proves the shift. It is consistent with it, and it says where the managing has to be done.

The fourth objection is the most serious. It comes from Rich Sutton's bitter lesson: that "general methods that leverage computation are ultimately the most effective, and by a large margin", and that "We have to learn the bitter lesson that building in how we think we think does not work in the long run" (Sutton, 2019). Rachel Laycock and her colleagues on Thoughtworks's Technology Advisory Board turned it on spec-driven development in their *Technology Radar* of November 2025: "We may be relearning a bitter lesson — that handcrafting detailed rules for AI ultimately doesn't scale" (Laycock et al., 2025, p.19). If the CELC asked organisations to write their context by hand, scale would make it redundant.

On the empirical point, Sutton and the Radar's authors are right. Handcrafted context does not scale. I have watched it fail on projects I have worked on, and the hands-on trials I know of point the same way. Birgitta Böckeler, who is also one of those authors, tried

Kiro and spec-kit herself and came away "very skeptical that lots of up-front spec design is a good idea"; faced with the output for a small bug, she would "rather review code than all these markdown files" (Böckeler, 2025). Colin Eberhardt, after "2,577 lines of markdown" and "3.5 hrs review time", was "a lot more productive without SDD, around ten times faster" (Eberhardt, 2025). That is two practitioners and a handful of tools, and they agree with the Radar. A year on, the Radar has moved a different kind of tool into Trial. OpenSpec is "a lightweight spec-driven development framework that works with any coding agent" that "centers on a spec-anchored workflow", and "Teams report more predictable agent output and fewer unintended changes compared to bare prompting" (Laycock et al., 2026, p.43). Where the spec persists alongside the code, it seems to fare better than a spec written once, up front. Context, though, does not have to be handcrafted. It behaves like data, and it can be handled like data: extracted, transformed and loaded. Sutton leaves room for exactly this. He writes that "we should build in only the meta-methods that can find and capture this arbitrary complexity" (Sutton, 2019), and the ETL of context is such a meta-method.

Most of the context a business needs, it already produces. Its day-to-day operations generate records that would exist anyway: tickets, documents, conversations, the trail its decisions leave. Until now, making those records usable by anyone else took an expert to interpret them, a person who stood as an obligatory passage point between the records and everyone who needed what was in them, and whose time limited how much got through. A model can now stand where that expert stood. It does not tire, does not take time off, and works at speed. That is a substitution.

What a business keeps its own hand on is narrower: the portion of context that carries its business rules, its compliance obligations, its brand and its taste, where it wants control over exactly what the model is given and where judgement stays with people. This portion should be curated by hand, because it is where the business makes sure its context is aligned with its goals, its mission and its vision: the governance of the CELC. It lives alongside the far larger body of context fed programmatically from the records of daily operations, and the two do not exclude each other. The bitter lesson holds for the handcrafted part, which is the smaller one.

The claim of succession rests on more than I have shown so far. To show why the SDLC cannot simply be run faster, I turn next to what the SDLC and its variants were for.

Every method is an opinion about complexity

The SDLC has never been one thing. It has been a succession of methods, each arriving with its own vocabulary and its own advocates: waterfall, which in hindsight was an early form of spec-driven development; the V-model, which paired each stage of specification with a stage of testing; Scrum, and the wider Agile family it belongs to; Extreme Programming, with test-driven development at its core; the twelve-factor app; DevSecOps. They are usually presented as process, as ways of organising work. I read each of them as an opinion: a view about how to stop the complexity of functional code from cascading past what the people responsible for it can manage.

The opinions differ on where the cascade can be stopped. Waterfall and the V-model stop it before the code exists, by deciding the system's parts and their connections up front and checking the result against that decision. XP stops it at the smallest unit, pinning each piece of behaviour with a test written before the code, so that a change which ripples further than intended shows up as a failure. Scrum and Agile stop it in time, by keeping each increment small enough to understand and cheap enough to throw away. The twelve-factor app stops it at the boundary between an application and the environment it runs in, and DevSecOps at the boundary between building software and securing and running it. Their advocates would describe them in other terms, as ways of serving customers, coping with requirements that change or shipping safely, and they would not be wrong. But a changed requirement ends as a change to code, and what it costs is set by how much of the code each change disturbs. Every method on the list is a bet about where that cost can be contained.

Each method also works best at a certain size, and grows heavier past it. A specification one architect could hold becomes a library nobody reads end to end, a test suite that ran while the kettle boiled runs overnight, and a stand-up for six becomes a ceremony for sixty. None of them fails all at once; each becomes overweight, as more of a team's effort goes into feeding it and less into what it was meant to protect. The usual response has been to cut the codebase down to a size the method can still serve, and I will come to where that response led. First, though, why the complexity grows faster than the code.

Start with the functional layer, code to code. Complexity lives in the edges between units, not in the lines of code. A thousand lines that call nothing else are easy to reason about. N functions, each of which could call, share state with or modify any other, give $N(N-1)/2$ potential pairwise interactions: 45 for 10 functions, 499,500 for 1,000. That growth is polynomial, quadratic in N, and it is the unconstrained worst case. Real dependency graphs are much sparser than that, and making them sparse is what architecture is for. In 1972 David Parnas argued that the criteria by which a system is decomposed into modules decide how modular it turns out to be, and proposed information hiding as the criterion (Parnas, 1972). The discipline that grew from it bounds each unit's connections, so that the number of edges grows close to linearly with the number of units. Complexity grows quadratically when nothing constrains coupling. Architecture is the discipline of constraining it.

Shared mutable state is worse. k flags or variables that interact give 2^k reachable states: 1,024 for ten flags, more than a million for twenty. This is where "exponential" is the right word, and it is the word I keep for it. Each new unit adds edges in proportion to the units already there. Each new interacting flag doubles the states.

None of that arithmetic is a measurement. It describes the shape of the risk, not its size, and the folk versions of it, which put a precise multiple on how much harder each new feature makes a system, claim more than anyone has measured. The defensible empirical frame is Meir Lehman's laws of software evolution, and his second law is the one that matters here: "As an evolving program is continually changed, its complexity, reflecting deteriorating structure, increases unless work is done to maintain or reduce it." (Leh-

man, 1980, p.1068). The weight of the sentence is on its last clause. Lehman does not say how fast complexity grows. He says it does not stop growing by itself, and that holding it back takes work, the work the methods were there to organise.

Code is also written and changed by people, and the people form a network of their own, with edges of their own. Before counting those edges, I want to ask what they carry, because the answer decides whether a lifecycle built on context is possible at all. Software is, among other things, a distillation of what those people understand of the problem it solves, and the most radical account of that understanding is Peter Naur's.

In "Programming as Theory Building" Naur argued that "the proper, primary aim of programming is, not to produce programs, but to have the programmers build theories", with documentation "an auxiliary, secondary product" (Naur, 1985, p.253). He then went as far as the argument can go. The theory, he wrote, "is something that could not conceivably be expressed, but is inextricably bound to human beings". He held that "program revival, that is reestablishing the theory of a program merely from the documentation, is strictly impossible", and that the death of a program "happens when the programmer team possessing its theory is dissolved" (p.258).

Anyone proposing a lifecycle built on context has to face that page. Agents act on what can be given to them, and Naur says that what matters most about a program cannot be given to anyone in writing. If he is right in the way he meant it, the CELC is an attempt at the strictly impossible.

I think Naur was right about how programs die and wrong about where their theory lives, and the way to see both is through a process that three traditions have named, each from its own side. The network that made something drops out of sight, and what is left is a thing that simply works. Historical materialism calls this "reification", after Georg Lukács, for whom "a relation between people takes on the character of a thing and thus acquires a 'phantom objectivity'", an autonomy that conceals "every trace of its fundamental nature: the relation between people" (Lukács, 1971). The stress there falls on the labour and the relations that are forgotten. Actor-network theory has two names for it. Bruno Latour's "black-boxing" stresses that the network becomes invisible because it succeeds: once a thing runs reliably, attention goes to its inputs and outputs and not to its insides (Latour, 1999). Michel Callon's and John Law's "punctualisation" stresses that a whole network comes to act as a single point (Callon, 1991; Law, 1992). My own thesis runs the first and the last together, calling a simplification "a reified understanding or in the context of this work, punctualized" (Li, 2020, p.68), and glosses the middle one as "intentionally making opaque the inner workings of technologies so as to reduce the complexity of comprehending them to inputs and resultant outputs" (p.87). They are not quite synonyms, but they name one process from different sides. Of the three, black-boxing is the one most readers already know, from engineering as much as from sociology, and it is the one I will use from here on.

Read that way, a program, its documentation included, is black-boxed theory. It works, and it can no longer be traced back to the thinking that produced it. The documentation

does carry some of the context that passed between the members of the team, but it is part of the box, not a key to it. Naur is right that the box cannot be reopened from its contents, or not at any cost worth paying, a point I return to later. Where I part from him is over who he thinks was inside. He places nearly all the weight on the people in the team, and leaves out everything else that took part.

Actor-network theory gives good reason to include the rest. Its practitioners work from a principle that Callon first named, "generalised symmetry", in which, as I put it in my doctoral thesis, "all of the actants and objects that lend effect to the construction, or translation of an assemblage or actor network, are treated as ontologically equal" (Li, 2020, p.152). People are among those things, but not the only ones. A programmer's theory of a program was never theirs alone. It was built on what they were taught, on the code they inherited, on the documents and tickets they read and the conversations they had, on the sensibilities of the people who worked on the problem before them.

Nor was it ever held apart from the things it was built with. What is a programmer without the hardware, the software and the language they work in? Languages, compilers, frameworks, build tools, environments and machines do not carry a team's intentions through unchanged. In Latour's terms they are "mediators", which transform what passes through them, as opposed to intermediaries, which carry it unaltered (Latour, 2005), and an actor, in his phrase, is "what is made to act by a large star-shaped web of mediators flowing in and out of it" (Latour, 2005, p.217, quoted in Li, 2020, p.71). Those mediators hold part of the context too, and when the team dissolves they tend to go with it, as versions move on and environments are retired. That is why the trace cannot be recovered. The theory is a property of the whole heterogeneous network, of the people, the code, the documents, the conversations and the tickets, of the tools they worked through, and now of the harnesses and models that sit among them.

A large language model makes the point almost too neatly. It is built, quite literally, on a network of knowledge external to it, and it comes close to being the quintessential "actor-network", a thing which, as my thesis glosses the term, "whether it is seen as a self-determining actor or a network of a confluence of effects is dependent on perspective, and it is not reducible to either just an actor or just a network" (Li, 2020, p.80). A model answers as a single actor, a black box in the most literal sense, though it was made, and works, as a network.

Read Naur's p.258 again with that in mind and most of it holds. He is right that a program cannot be revived "merely from the documentation", because documentation is one element of the network, and revival from it alone asks one element to do the work of all of them. He is right that programs die when their teams dissolve. What he describes as death is a network coming apart: the human links that held it together are cut, the mediators they worked through are lost with them, and what remains no longer adds up to a theory of anything. It comes apart for a reason Naur did not name. Once a program is black-boxed it is necessarily precarious, because the network inside the box stays aligned only for as long as somebody keeps it aligned. The box looks stable, though, and that stability is an illusion which discourages the very maintenance it depends on. Nor is the box

inert in this: simply by working, it argues that nothing needs doing. So when a link inside it does fail, in a gap where any substitute is shallow, there is nothing left to repair it from. The program becomes unrecoverable, which is Naur's death, re-described.

The one claim that does not hold is the one that binds theory to human beings alone, and it hides a blind spot, because a program can die from the non-human side as well. The familiar case is a dependency that stops being maintained, or that updates in a way that breaks the program around it. Models and harnesses are mediators too, and just as perishable, deprecated by their vendors or changed under the people who rely on them. Whether that is the old risk or a new scale of it is a question I come back to when I turn to how context ages. The CELC, on this reading, is not an attempt to write the theory down, and Naur is right that it cannot be written down in the sense he meant. It is the practice of keeping the network that holds the theory assembled, with context and models among its members, and of treating the loss of any member, human or not, as a standing risk to be built for.

In practice this is the division I drew earlier, in answering the bitter lesson. A model now does what the expert mediator did: it interprets the records a business produces in its daily operations and renders them usable by everyone else. Human judgement stays obligatory in the narrow, governed portion, in the business rules, the compliance obligations, the brand and the taste that a business keeps its own hand on. This describes where the network routes certain decisions, not where its theory resides, so it concedes nothing to the claim that theory lives in people alone. People matter to a program's theory, as Naur said, without being the only things that hold it.

Back, then, to the edges. The team that holds a program's theory is also a network in the plainest arithmetical sense, and Fred Brooks did the sum. Developers who each need to talk to every other have $n(n-1)/2$ communication paths between them: one for two developers, 45 for ten. That overhead, together with the time newcomers take to learn what the others already know, is what sits behind his law: "Adding manpower to a late software project makes it later" (Brooks, 1975). It is the same unconstrained worst case as on the functional layer, and it is cut down the same way. Team structures, the interfaces between teams and the documents that pass across them exist so that most people never need to talk to most others. They are information hiding for people.

The two networks do not stay apart. Melvin Conway's observation, which Brooks later named Conway's law, is that "organizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations" (Conway, 1968). Teams that do not talk produce interfaces that do not fit, and the glue code at the seams is where the gaps in the social network show up in the functional one.

For a long time that was an engineer's proverb. It has since been tested. Alan MacCormack, Carliss Baldwin and John Rusnak compared matched pairs of products from tightly and loosely coupled organisations, the loosely coupled ones open-source, and found the loosely coupled organisations' products significantly more modular, by up to a

factor of six in how far a change propagates (MacCormack et al., 2012). Nachiappan Nagappan, Brendan Murphy and Victor Basili found, on Windows Vista, that measures of the organisation predicted which parts of the system would fail better than the code metrics did, churn, complexity, coverage and dependencies among them (Nagappan et al., 2008).

Put the two layers together and the trouble compounds. A dense social network laid over a dense functional network asks more of the people in it than anyone can carry. No developer can hold the system's state in their head, and the effects of a single change spread combinatorially, along functional edges and social ones at once, faster than any one person can follow. As I put it in my thesis, about a different kind of network, "the human mind is often unable to detect network complexity, much less comprehend it on an infinite scale" (Li, 2020, p.86). What follows is familiar to anyone who has looked after a large estate: technical debt, bugs and deployments that come further and further apart. Every method I listed at the start is, in effect, a strategy for keeping one or both of these networks sparse.

When the methods grew overweight, the answer was to cut the system up until each piece was small enough for a method to serve again. That is where microservices came from: a service per team, sized so the team could hold it, behind an interface the other teams could rely on without looking inside. Naur's programs were black-boxed by default, as a side effect of the work; a microservice is black-boxed by design, its insides hidden so that everyone else can treat it as inputs and outputs. For each team, for a while, it worked.

Then the boxes multiplied. Callon's word fits what each service became: "The process of punctualisation thus converts an entire network into a single point or node in another network" (Callon, 1991). Each service was a network of code, people and decisions folded into a node, and the estate was a network of those nodes. The arithmetic of the functional and social layers did not go away. It moved up a level, from functions to services and from developers to teams, and it moved to where fewer people were looking. In the engineering organisations that went this way, each team could see inside its own box and the interfaces of its neighbours, and nobody could trace the network the boxes made between them. Adding architects to watch over it did not bring the view back, because each of them watched from inside a part. The risks the methods had been meant to remove returned at the seams, where nobody owned them.

In actor-network terms the black-boxed state is "highly precarious", carrying "the risk of subversion, and the encapsulated network of associations degenerating into a mess of naked, failing networks" (Li, 2020, p.87). It is a fair description of a large microservice estate on a bad night.

Some organisations have since pulled their services back together, into fewer of them and into a single repository. It is often read as the pendulum swinging back, monolith against microservice, as though the argument had been about how many boxes to have. I do not think it ever was. What consolidation bought, where it worked, was an architec-

ture people could understand again: fewer boxes, fewer seams, a network somebody could trace from end to end. Consolidation was the means to understandability, and understandability had been the objective of every method on the list, from waterfall's specification and XP's tests to Scrum's small team and the microservice's small service.

If every method was, in the end, a way of keeping software understandable to the people responsible for it, then the question for a lifecycle of generated software is not how quickly it produces code. It is where the complexity goes once people no longer write the code, and what keeps it understandable when it gets there. I will argue that it goes into context. Before that, there is an older story to tell, about what happens to the organisation of work when the thing that powers it changes.

From the craft shop to the dynamo

The thing came before its name, and for a while it was named for what it was not. "Automobile" reached English from French, built from the Greek *autos*, self, and the Latin *mobilis*, movable. Before that word settled, the thing was a motor carriage, a steam carriage or a horseless carriage: a carriage with the horse taken away. Henry Ford's first, which ran in 1896, was the Quadricycle, named for its four bicycle wheels, and Ford called it his "gasoline buggy" (Ford and Crowther, 1922). By 1922, when he wrote his memoir, the horseless carriage belonged to the years in which it "was considered merely a freak notion" (Ford and Crowther, 1922).

We are in the horseless-carriage years of software. Look again at the vocabulary I sorted at the start: AI-assisted development, the AI-Driven Development Life Cycle, the agentic SDLC. Each names the new thing as the old thing with something added: development, with assistance; the lifecycle, with AI in front of it. I use "AI-assisted development" myself, for the practice as it stands, because that is what it still is. A name like that records what its users still take for granted, as the horseless carriage took the carriage for granted.

The name lagged the thing. So did the organisation of the work around it. I will read the history of the motor industry the way a historical materialist reads any history of production. Each of its four stages, craft, Ford, Sloan and Toyota, was a change in the forces of production, the tools, machines and methods with which the work was done, and each eventually reorganised the relations of production: who does the work, who controls the process and what counts as skill. I give each stage its due, because AI-assisted development is a change of the same kind, and the question is which reorganisation it brings.

Before Ford, cars were hand-built in small numbers by skilled craftspeople. Parts made in different shops did not quite fit, so fitters filed them down until they did; in the case James Womack, Daniel Jones and Daniel Roos describe, the body came from a Paris coachbuilder; no two cars were exactly alike; and most of the workers had come up through an apprenticeship (Womack et al., 1990, pp.21–26). The relations of production followed from the forces. The skilled worker held the knowledge of how the thing went together, set the pace at which it did, and was the one who could say whether it was right.

Skill meant fitting, the trained judgement that made this part meet that one. The product could be superb. It was also slow and expensive, and it depended on people who had served apprenticeships.

The traditional SDLC is the craft shop. Engineers write code line by line, fitting bespoke logic, architectures and workarounds to each application, and the theory of the thing lives in the network of people and tools I described above. The SDLC was craft at the bench and Ford in the office: code fitted by hand, inside a process divided into stages and handed down. Craft will persist, and it should. For the gnarliest problems, the ones nobody has solved before, the craft shop can still build the best possible vehicle for one particular purpose, and the engineers who can work that way will be well paid for it. It will not be how most software is made. Craft manages its unit, functional code, at a finer grain than the work now needs, once much of that unit can be generated. Choosing not to automate, or to automate less, is now an opinion, not a requirement, which puts craft in the same company as every method on the list above.

Ford changed the forces of production by breaking the fitting into steps. In his own account, the best his plant had done at stationary assembly was "an average of twelve hours and twenty-eight minutes per chassis". Pulling the chassis down a line by rope and windlass cut that to "five hours fifty minutes", and in early 1914 a raised line and "a further subdivision of work so that each man had fewer movements cut down the labour time per chassis to one hour thirty-three minutes" (Ford and Crowther, 1922). Those are hours of labour per chassis, not the time it took to build a car: what fell was the human labour in each one. The assembly of the motor, "formerly done by one man, is now divided into eighty-four operations" (Ford and Crowther, 1922), and that division of labour is how a luxury became a mass product.

Ford was frank about where the skill went. The principles of the line, he wrote, produced "the reduction of the necessity for thought on the part of the worker and the reduction of his movements to a minimum", so that the worker "does as nearly as possible only one thing with only one movement". He denied that he had taken skill out of work: "We have not. We have put in skill. We have put a higher skill into planning, management, and tool building, and the results of that skill are enjoyed by the man who is not skilled" (Ford and Crowther, 1922). Skill moved off the floor, into the offices that planned the line and the tool rooms that built it, and whoever designed the line now set its pace. The best-known line in the memoir belongs here too: "Any customer can have a car painted any colour that he wants so long as it is black." Ford placed the remark in 1909, writing in 1922, and Model Ts had in fact come in other colours until about 1914. It is a founder's polished retrospective, and still a fair summary of what the line did to the product: one chassis and one model, made one way.

Alfred P. Sloan's General Motors beat Ford by refusing the one model. Sloan's policy, in his own words, was "a line of cars in each price area, from the lowest price up to one for a strictly high grade, quantity production car" (Sloan, quoted in Tedlow, 1988, pp.55–56), and the range "came to be known as 'the car for every purse and purpose'" (Tedlow, 1988, p.56). GM shared parts beneath its brands and changed its models every year, though

"The annual model change… was the innovation no one wanted" (Tedlow, 1988, p.56): flexible mass production, in place of one chassis for every customer. Ford paid for waiting. "Ford's market share collapsed to under 10% in 1927" (Tedlow, 1988, p.51). On 26 May that year Henry and Edsel Ford drove the fifteen-millionth Model T out of the Highland Park plant, which ceremonially ended its production (The Henry Ford, n.d.a). The changeover to the Model A meant "the replacement of some 15,000 machine tools, the total rebuilding of another 25,000" (Sward, quoted in Tedlow, 1988, p.51), and the new car was introduced on 2 December 1927 (The Henry Ford, n.d.b). The forces of production had changed again, and the relations changed with them. Variety had to be planned, timed and paid for, and what counted as skill moved again, towards the people who could manage variety at scale: who decided the range, which parts could be shared and which had to differ, and when each model would change.

AI-assisted development went from Ford to Sloan almost at once. The arrangement I described at the start, with the rendering done in the user's harness, is already Sloanist. A foundation model is the shared chassis. On top of it, variations are prompted, configured and skinned for niches, for departments and in the end for individual users, and rendered in whichever harness the user works through. That is mass customisation: the speed of mass production without giving up the tailoring, down to the person at the far end of the wire. Nobody selling software now would say the equivalent of Ford's line about black. The skill that rises in value is Sloan's, the management of variety, and it sits, as it did at GM, with whoever controls the chassis.

Toyota changed the forces of production in another direction. The Toyota Production System, as Taiichi Ohno set it out, stood on two pillars: just-in-time, making only what is needed when it is needed, and *jidoka*, usually rendered in English as autonomation, in which machines and lines are built to stop at the first abnormality so that its cause can be found and fixed (Ohno, 1988). Any worker on the line could stop it on seeing a defect. Quality was built in where the work was done, instead of inspected in at the end.

Read as relations of production, this is the most interesting of the four stages, because it moved some control back onto the floor. A worker who can stop the line holds an authority over its pace that Ford's worker never had. A line that stops at each abnormality also learns from each one, since every defect becomes the occasion to find its cause and change the process: a loop that runs from the floor back into the design of the work. This is the stage I keep: the learning loop and built-in quality are what the CELC takes from the motor industry, and I will come back to both.

Toyota did not travel as Toyota, though. In the West it travelled as lean, and on the way it changed. Mike Parker and Jane Slaughter, writing from the American plants that adopted Toyota's team concept, called what they found there management-by-stress: a system kept under constant pressure, in which any slack is found and removed. In their words, "the operating principle of management-by-stress is to systematically locate and remove such protections", and "The system, including its human elements, operates in a state of permanent stress" (Parker and Slaughter, 1989; see also Parker and Slaughter, 1988). Read that way, the pressure crossed the Pacific more reliably than the authority of the stop

cord, or the learning it served. Lean, as the West repositioned it, meant the intensification of work and the cutting of costs, and in that form I refuse it.

Actor-network theory has a word for what happened. In my doctoral thesis I put it this way: "How an actor-network expands, influences, consolidates and defends itself is through the process of 'translation'", in which "the associations are negotiated, and always in danger of succumbing to oppositional effects, resulting in 'translation error'; success is finite, always subject to movement" (Li, 2020, p.82). Translation is not transmission: a method that moves from one network into another is reassembled there, out of whatever the new network will hold. Lean is Toyota translated into the networks of Western management, and the authority of the stop cord was the part those networks would not hold. That is a translation error in the sense just quoted. A historical materialist would put the same point structurally: a technique that moves between two sets of relations of production is adopted on the terms of the relations that receive it. Here the two lenses I named at the start pull against each other, as I said they sometimes would. Actor-network theory refuses to treat a structure as given before the network that performs it; historical materialism begins with the structure. On lean they disagree about why, and agree about what happened.

The same translation is under way again, with AI-assisted development as the technique and the same networks receiving it. Read what the people building and selling the tools say, in their own dated words. AWS warns that "Simply retrofitting AI as an assistant not only constrains its capabilities but also reinforces outdated inefficiencies", and offers in place of sprints "shorter, more intense work cycles" (AWS, 2025). Anthropic's researchers ask, as an open question, whether developers will "shift to mostly managing and guiding these systems, rather than writing code themselves" (Anthropic, 2025). Karpathy's stated goal is "to claim the leverage from the use of agents but without any compromise on the quality of the software" (Karpathy, 2026). None of these sentences says cut costs, and the last explicitly puts quality first.

Others are franker. Asked by an analyst about "labor arbitrage" in February 2025, Salesforce's chief executive, Marc Benioff, said: "I would say that we definitely have seen a lot of efficiency with engineering… And we're not going to hire any new engineers this year. We're seeing 30% productivity increase on engineering." (Benioff, 2025). Amazon's chief executive, Andy Jassy, wrote in August 2024 that its coding assistant, put to work on Java upgrades, had "saved us the equivalent of 4,500 developer-years of work (yes, that number is crazy but, real)", with the upgrades improving security and reducing infrastructure costs, "providing an estimated $260M in annualized efficiency gains" (Jassy, 2024). Ten months later, writing to staff about generative AI across the whole company, not only its engineers, he was plainer: "We will need fewer people doing some of the jobs that are being done today, and more people doing other types of jobs", and "in the next few years, we expect that this will reduce our total corporate workforce" (Jassy, 2025). Every figure there is a company's estimate of itself. Even the frank ones mostly speak of efficiency, productivity and hiring, and leave the arithmetic to the reader. The consultancy does the arithmetic. BCG's April 2025 deck for executives promises "5-10% typical cost savings for

CIOs", foresees "Fewer layers, faster releases, leaner headcount", and, among its levers for driving adoption, lists "Reduce engineering budgets or headcounts to incentivize GenAI usage" (BCG, 2025, pp.3, 10, 15). That is scarcity used as an incentive, and lean would recognise it. Read the whole set through lean's history and the vocabulary is familiar: inefficiency to be removed, cycles made more intense, leverage to be claimed, engineers not hired, headcount made leaner. That vocabulary lands where lean's did, with the leaders, technical and not, who decide what the time it frees is for.

The word that does the work is "productivity". In my honours thesis I followed Sidanius and Pratto in reading meritocracy as a legitimising myth, and the move that matters here is theirs. Whether a belief "is epistemologically false or inaccurate, and consequently, whether or not it is valid, has nothing to do with the belief's power to legitimize inequality"; what sustains it is "the intensity of the person's belief in the truthfulness of the ideology" (Li, 2011, p.28). Productivity works the same way in our industry. The evidence I began with shows how unsettled its measurement is, and the most telling result in it was a belief: METR's developers "expected AI to speed them up by 24%, and even after experiencing the slowdown, they still believed AI had sped them up by 20%" (METR, 2025), in a study whose headline figure METR has since called "out of date" (METR, 2025; METR, 2026). A belief that survives its own measurement does not need to be true to do its work.

Harry Braverman said what that work was, half a century before agents. Writing about Taylorism, he argued that management under monopoly capital proceeded by separating conception from execution: the knowledge of how the work is done was taken from the people doing it and lodged with the people who planned it (Braverman, 1974). Ford had said the same thing more cheerfully, with skill put into "planning, management, and tool building" and taken off the floor. Productivity was the name under which that separation passed for common sense, and its power never rested on the numbers. AI-assisted development poses the question again. Execution passes to agents. What is left to people is conception, judgement and oversight, and the question is whether that is treated as the work itself or as the remainder of a cost not yet cut.

So AI-assisted development intensifies work, as lean did, and it does so differently. Lean intensified by filling the slack with more of the same work. Agents intensify by taking away the easy work: the boilerplate, the reuse of functions already written, the implementation of logic somebody else has already decided. That work took little thought, and it was where people recovered between the parts that took a lot. What remains is decision and oversight, all day. Lisanne Bainbridge saw the shape of this in 1983, writing about the automation of industrial processes: "the designer who tries to eliminate the operator still leaves the operator to do the tasks which the designer cannot think how to automate", and the monitoring that remains is "very boring but very responsible". Her verdict: "The human monitor has been given an impossible task." (Bainbridge, 1983, pp.775–776). Karpathy's working day, in which "you are orchestrating agents who do and acting as oversight" (Karpathy, 2026), is that job, and he calls it "the new default".

The first measurement of what that costs people came in March 2026, when Bedard and colleagues reported, in the *Harvard Business Review*, a survey of 1,488 full-time workers

in the United States, about 14% of whom reported symptoms of what the authors call "brain fry". Among workers whose jobs involved high levels of AI oversight, reported mental effort was 14% higher, mental fatigue 12% higher and information overload 19% higher (Bedard et al., 2026). There are caveats. It is a self-reported survey, run by BCG, which sells AI consulting; "brain fry" is not a clinical term; and the sample is workers in general, not developers. It is a first sign, and it points where Bainbridge pointed: the load concentrates in the oversight.

The closest analogy I know is the assembly line itself, a generation on. The worker who used to insert parts into a chassis now supervises robots that insert them at superhuman speed and do not tire. The work is no lighter: at line speed a mistake cascades faster, through more units before anyone sees it, and every stoppage, planned or not, has a larger blast radius, because more is in motion when the line stops. The effort and the time pressure concentrate in the oversight and the decisions, which is where the human was left, and the speed makes Toyota's cord harder to pull, since each pull now stops more.

The same tool, then, widens a range with two ends. At one end the time it frees goes into better work, in the same time or less: more of the output reviewed and understood, quality built in where the work is done. At the other, the freed time is taken as savings: fewer people, the same deadlines, and the oversight piled on whoever is left. The honours thesis has names for the two readings of productivity that go with them. As I glossed Sidanius and Pratto there, the distinction is between "Hierarchy-Enhancing (in other words, legitimizing myths which support inequality) and Hierarchy-Attenuating (those that support equality)" legitimising myths (Li, 2011, p.29). Productivity as cost-cutting is the hierarchy-enhancing myth: it moves conception further from execution, and the gains further from the people doing the work. Productivity as better work in the same time is the attenuating one. Cost-cutting is the reading the pitch most readily invites, and some leaders have already taken it: in February 2026 *Fortune* reported a finding that "37% of organizations plan to replace early career roles with AI" (Fore, 2026). It is still a choice, because people decide where the tool lands, and the CELC puts it at the quality end, managing the cognitive load of oversight instead of leaving people to absorb it. The attenuating reading is a myth in the same technical sense, but it can be checked, in review queues and in the people at the end of them, and the CELC measures both.

One question I will raise here and not argue. If juniors learned their trade on exactly the work agents now do, the boilerplate, the small fixes, the logic someone else had already decided, how does the next generation become mid- and late-career engineers? Is the ladder being pulled up for good? The evidence so far is early and mixed. Employment of 22- to 25-year-olds in AI-exposed occupations stands 19% below its counterfactual (Brynjolfsson et al., 2026a), though the same authors stress that "the precipitous decline in employment for young software developers is a consequence of various factors" (Brynjolfsson et al., 2026b). At Big Tech, SignalFire found that new graduates "now account for just 7% of hires, with new hires down 25% from 2023" (SignalFire, 2025). And IBM's chief human resources officer said in February 2026: "We are tripling our entry-level hiring, and yes,

that is for software developers and all these jobs we're being told AI can do" (LaMoreaux, quoted in Fore, 2026). I leave the question open.

One more episode in the history of production bears on all of this. In 1987 Robert Solow, reviewing a book for the *New York Times Book Review*, remarked that the computer age could be seen everywhere except in the productivity statistics (Solow, 1987). Three years later the economic historian Paul David answered with a precedent. In 1900, he wrote, observers "well might have remarked that the electric dynamos were to be seen 'everywhere but in the productivity statistics!'" (David, 1990, p.356). Factory electrification did not "have an impact on productivity growth in manufacturing before the early 1920s… This was four decades after the first central power station opened for business" (David, 1990, p.357).

David's explanation for his period was material. The delay, he argued, came from "the unprofitability of replacing still serviceable manufacturing plants" (David, 1990, p.357). A factory built around a steam engine was built around its line shaft: one engine turning one long shaft, with every machine in the building belted to it, and, because turning very long shafts wasted power, the building itself raised over several storeys to keep them short (David, 1990, p.358). Nobody tears that down while it still works. So for a generation manufacturers compromised. The "'group drive' system", in which electric motors drove groups of machines through shorter shafts, stayed in vogue from the mid-1890s to the eve of the 1920s (David, 1990, pp.357–358). The gain came only with unit drive, a motor for each machine, which freed the layout. Factories could be single-storey and arranged around the flow of the work, not the reach of a shaft (David, 1990, p.358), and about half of the acceleration in total factor productivity in manufacturing between 1919 and 1929 is statistically accounted for by the growth of that secondary motor capacity (David, 1990, p.359). For 1900, that is the right account, and a historical materialist should be glad of it.

The material conditions of this transition are different, and so, I think, is the cause of its lag. Capital is not what is short. In June 2026 Morgan Stanley forecast that AI-related global debt issuance would "more than double to nearly $570 billion in 2026", with four hyperscalers "expected to spend $700 billion in outlays this year" and hyperscaler capital expenditure passing "$1 trillion in 2027" (Reuters, 2026). Capital may be going to AI faster than to any technology before it, enough to draw on money that would otherwise sit in the safe havens; I would put it no more strongly than that. The nearer precedent is not the dynamo but the internet, and the capital sunk into the infrastructure that carried it: dramatic overcapacity, and bets on businesses that were flimsy at best. At the time nobody knew how much capacity would be needed, and the only position anyone took was not to have too little and be left behind. Fear of missing out is a strong motive for companies and their investors. It is only sometimes an existential one.

Capital and material conditions do return, but as a risk on the horizon, not as the cause of the lag today. The most advanced chipmaking is concentrated in a handful of places: in 2021 all of the capacity "in nodes below 10 nanometers" was in "South Korea (8%) and Taiwan (92%)", which the report's authors called "single points of failure" (Varas et al.,

2021, p.5). A conflict in East Asia, or a scaling limit in this generation of hardware, could create a material crunch. There are countervailing forces, among them alternatives to Nvidia's chips and models that are starting to optimise the code their own hardware runs (Ouyang et al., 2025; Li et al., 2025), but they are early, and these risks cut both ways.

The resistance is real as well, and it is already showing up in public. In the United States midterm elections of November 2026, local communities "opposed data center-related ballot measures in 40 out of 41 cases" (Capoot, 2026). Add slow training and slow adoption inside organisations, and it is tempting to call all of this the material brake. In my view, unqualified as it is, it may be partly a symptom of the same thing as the rest, and not only an impediment of that kind: a lack of imagination about what the technology is for, beyond the data centre that is to be built down the road.

That is where I place the lag. Its cause is mainly a failure of imagination, and the reorganisation it holds up is wider than ways of working. It runs through the infrastructure of society, business, government, media and technology, all of which were built around the old forces of production and all of which have to be rebuilt around the new. That will take longer than the accelerationists expect, and it will probably be faster than the doomers would have hoped. The historical evidence is on the side of speed. Diego Comin and Bart Hobijn, tracing fifteen technologies across 166 countries over two centuries, found that "on average, countries have adopted technologies 45 years after their invention", and that "Newer technologies have been adopted faster than old ones" (Comin and Hobijn, 2010). Generative AI fits the trend. By late 2024 "nearly 40 percent of the U.S. population age 18-64" was using it, and "overall adoption has been faster than either PCs or the internet", though at work it has been only "as fast as the personal computer" (Bick et al., 2024). By the measures we have, this is on track to be the fastest technology transition yet. But what has been fast is the adoption of the tool, and adoption is not reorganisation. A tool can be everywhere while the ways of working around it stay as they were, and that gap is the lag. The CELC is that reorganisation, for ways of working.

A historical materialist should be uneasy calling the cause of a lag imagination, and I am, a little. But the imagination I mean is not a free-floating idea. It is the capacity of people and organisations formed under one set of relations of production to picture another, and it is limited by the relations it has to see past. In David's factories the forces of production outran the relations because the old plant was too valuable to scrap. In ours they outrun them because the old organisation is too familiar to see around.

The misstep, while that lasts, is to treat AI as a better steam engine inside the old line shaft. In software the line shaft is the code layer, and the sprint is the rhythm it turns at: one rhythm to which every piece of work is belted, with its planning, its review and its release turned at the same speed. Using agents to write code faster inside a standard two-week sprint is group drive, a new motor on the old shaft. AWS's "shorter, more intense work cycles" (AWS, 2025) spin the shaft faster. What happens next is what the evidence I began with already shows: the code piles up faster than review and deployment can absorb it. DORA warns that "acceleration can expose weaknesses downstream" (Harvey and DeBellis, 2025), and Faros, a vendor whose telemetry is correlational and should be read

that way (it compares periods of low and high adoption within each organisation, across some 22,000 developers), reports that "Median time in review is up 441.5%" (Faros, 2026).

The reorganisation is to take the shaft down: to give each unit of work its own motor, and to arrange the work around its flow instead of around one shared rhythm. In software, unit drive corresponds to enabling swarms of agents. In its most ambitious form, that means an engineer directing an army of micro-agents, each with its own feature, its own tests and its own deployment loop. We already have the proliferation; the problem it brings is oversight and coordination: of the humans, of the agents, and arguably of the platforms, networks and data underneath them, which have never been well coordinated or well monitored in general. The microservice estate shows the risk. When services multiplied, the risks the methods had been meant to remove returned at the seams, where nobody owned them, and a fleet of agents has more seams than any estate.

There is no consensus yet on how to coordinate it, only a spectrum of approaches. At one end is complete control: a central orchestrator that delegates every task and checks every result, which is hard to achieve over a system that is probabilistic all the way down. At the other is laissez-faire, teams that let 'er rip, and report figures so outlandish that they read as unbelievable. At AI Engineer Europe in April 2026 Vincent Koc, one of OpenClaw's core maintainers, described a project where "at the peak, we were doing 800 commits a day... there's about 10 to 15 core maintainers all with day jobs", a personal day "Where I hit close to 3,000 commits per day", and a method: "So, we did commit maxing... Smash as many commits as we can." (Koc, 2026). Writing for OpenAI in February 2026, Ryan Lopopolo described an internal beta built "with 0 lines of manually-written code", in which "roughly 1,500 pull requests have been opened and merged with a small team of just three engineers", and where "Humans may review pull requests, but aren't required to." (Lopopolo, 2026). For scale, Nick Arcolano of Jellyfish, looking across some 20 million pull requests, found that going from no use of AI to full adoption roughly doubled the pull requests per engineer (Arcolano, 2025). Doubling is a large effect, and a long way short of three thousand commits in a day. Yubin Kim and Xin Liu's experiments show what is at stake at each end, without settling which is right for any given organisation: agents working in parallel without talking to each other amplified errors "by up to 17.2x", a central orchestrator "contained this amplification to just 4.4x", and on tasks that had to be done in order "every multi-agent variant we tested degraded performance by 39-70%" (Kim and Liu, 2026). Where most organisations land will probably be somewhere in between. Where any one of them should land is a question of method, with its trade-offs to weigh, and this essay sets the frame for it and leaves it open.

David himself warned against pushing his analogy too far. "Computers are not dynamos" (David, 1990, p.360), and agents are not electric motors. The dynamo supplied one undifferentiated thing, power, and once each machine had its own motor the factory could be rebuilt around the flow of its work. What agents supply is less uniform, and what the reorganisation has to be built around is less obvious. The analogy gives the shape of the change, a technology already everywhere, waiting on a reorganisation of the work around it. It does not give the timing, and it does not say what to build around.

Freed from the shaft, the factory was rebuilt around the flow of its work. Freed from writing code, a lifecycle has to be rebuilt around what now flows through the work, and what has to be kept understandable within it: context. Every method, as I argued above, was a way of keeping things understandable. What follows argues that the complexity is now mirrored in context, and asks whether any of it vanishes on the way.

Where the complexity went

The history I have just told is a historical materialist's, and it explains a good deal: why a change in the forces of production ends in a reorganisation of the relations of production, why the reorganisation lags, and which way the gains are likely to run while it does. This section earns the claim I staked at the start, that essential complexity is mirrored in context; first, a word on the lens it uses. What it explains less well is how the reorganisation happens inside any one organisation: which people, documents, tools and now agents are drawn into the new arrangement, in what order, what holds them there and where it comes apart. A history at that altitude can say that factories would be rebuilt around unit drive. It cannot say why one plant's rebuild worked and another's stalled, and it is not meant to. Actor-network theory is meant for little else. From here I lean on it, with the material history underneath as the frame.

At lean I said that the two lenses disagree about why and agree about what happened. Since what follows leans on one of them, the disagreement should be stated in full. Actor-network theory is flat. It grants no structure a reality prior to the network that performs it: no class, no market and no organisation chart standing behind the associations and explaining them. The flatness is deliberate, and it is what lets the theory follow a compiler or a model as seriously as a person; as I wrote in my doctoral thesis, "its ontological flatness allows the incorporation of methods that cross intellectual boundaries" (Li, 2020, p.88). It also has a consequence, which the same thesis put plainly: "Power, therefore, is inter-connectivity. One actor-network cannot be said to be 'more powerful' than another, rather, it is simply bound to more associations" (Li, 2020, p.82). A historical materialist will answer that this is precisely what a flat theory cannot see: why some networks come to be bound to more associations than others. The $26.3 billion that Google paid to be the default in the antitrust case I cited at the start was not spent on associations at random. Historical materialism begins with that structure, with who owns the passage point and what bought it, and reads the associations as its effects.

Actor-network theory has its own answer, which is that size is made. "All differences in level, size and scope are the result of a battle or a negotiation", as Callon and Latour put it; "An actor grows with the number of relations he or she can put, as we say, in black boxes", and so "macro-actors are micro-actors seated on top of many (leaky) black boxes" (Callon and Latour, 1981, pp.279, 284–285, 286).

Much of that growing is now paid for. What Google's payments bought were associations: the placements, and with them the habits of everyone who never changed a default. Historical materialism says why the money was spent and whom it was spent for. Actor-network theory follows it into the network and shows why what it bought stays precarious.

The boxes leak, and those an actor enrols "can desert it" (p.289). What money enrols, money has to keep enrolled. Read that way, the two lenses stop competing, because they are doing two halves of the same work. Naming the interest and tracing the associations are two halves of one act.

Most of the vocabulary the rest of this needs is already in place. Black-boxing, mediators, generalised symmetry and translation were each introduced where the argument first called for them, and I use them here as glossed there. One more term is needed before going on. The theory's most general unit is the "actant", which in Latour's definition "can literally be anything provided it is granted to be the source of an action" (Latour, 1996, p.373). The corollary, as my thesis put it, is that "an object, an actant, an actor-network, can only be defined by its associations with other objects, actants, actor-networks" (Li, 2020, p.80). The word commits to nothing about the thing that acts, neither whether it is a person nor whether it means to, and that is why I use it: it lets a coding agent into the account without first settling whether the agent is a tool or a colleague, an argument this essay has no need to win.

Under generalised symmetry, as I described it earlier, agents are actants in exactly this sense. They open pull requests, change code, call tools and write records that other people and other agents then act on, and each of those is an effect somebody downstream has to answer to. In practice the corollary does more work than the definition. If an actant is defined by its associations, there is no such thing as "the agent" apart from the model it runs on, the harness it runs in, the tools it is allowed to call and what it has been given to work from. Change any of them and you have changed what is acting, whatever the dashboard still calls it. "Is this agent good at the task?" is, on this reading, a malformed question. The answerable one is whether this assembly is good at it: this model, in this harness, with these tools, given this material. The same goes for the people. An engineer reviewing a pull request with the ticket in front of them is not the same actant as the same engineer reviewing it without.

Translation, the term I took up for lean, names how such assemblies form: a method that moves from one network into another is reassembled there, out of whatever the new network will hold. AI-assisted development is being translated into software organisations now, and the sections above have set out what it is being translated into: the theory of a program spread across a network of people and things, a complexity that every method on the list was a bet about containing, and an organisation of work that lags its tools.

At the start I stated the claim I am now obliged to earn: that generating code does not make essential complexity disappear; rather, it is now mirrored in the context that directed the generation of code. I also left two questions hanging: whether some complexity does in fact vanish on the way, and what stops context, once it carries the complexity, from fragmenting the way code did. Both come due here.

Take the first. Some complexity does vanish. Brooks never held that all of software's difficulty was essential. Much of it was accidental, belonging to the tools rather than the problem, and he credited high-level languages with stripping a good deal of that away

(Brooks, 1987, p.12). Generation strips away more, the boilerplate, the glue and the bespoke how, and that part is gone; it was never the part I claimed is conserved. The essential complexity neither vanishes nor leaves the code: it is mirrored in context, and context is where it is now acted upon. In Latour's terms, code has become an intermediary, carrying that complexity without transforming it, and context is the mediator, where people and agents transform it, question it and change it. The complexity was always there; what has moved is the acting upon it. Generating code does not destroy complexity. It migrates. Lehman even supplies the word, though he used it for something else: his fifth law is titled "Conservation of Familiarity (Perceived Complexity)" (Lehman, 1980, p.1068). Context left unmanaged is simply the absence of a CELC, and unmanaged context behaves like an unmanaged codebase. Lehman's second law, which I quoted for code, applies to it without amendment: its complexity "increases unless work is done to maintain or reduce it" (Lehman, 1980, p.1068), and it grows the way code's did, faster than the context itself, along the edges between its parts.

The signs are consistent with it already. The same *Technology Radar* that moved OpenSpec into Trial puts AI-driven documentation sprawl under Caution, warning that "repositories rapidly accumulate verbose, unmaintained and contradictory specs that developers lack the time to clean up, resulting in a chaotic 'big ball of documentation mud'" (Laycock et al., 2026, p.27). It is Lehman's law again, applied to context: context that nobody maintains is not maintainable, any more than code that nobody maintains ever was.

The other signs are the ones I set out at the start, and they point to the same place. The bottleneck has moved from making the code to reviewing it, overseeing it and putting right what gets through. Faros's review figure is the same 441.5% rise, and Faros also reports that "The incidents-to-PR ratio is up 242.7%", which is, as Faros itself says, "a ratio, not a probability" (Faros, 2026). GitClear's clones are more code for somebody to read (GitClear, 2025), and Bainbridge's monitor, as I described above, has been left the job that remains: understanding code that has become cheap to make.

There is a bottleneck before generation too, and it is not new. Gathering the context a piece of work needs, meaning what the business wants, what has already been decided and what the estate will and will not bear, has always been the first gate, whatever the method called it: waterfall's requirements, Scrum's backlog refinement, the conversation that an XP story card was only ever a reminder of. It remains the first gate, though who waits at it has changed. An engineer who missed something at the gate could recover it in the corridor. An agent acts on what it has been given. Gathering context is where, in Naur's terms, the theory of a program begins to be built, and on the reading I gave of him that theory is built across the whole network. So what an organisation needs is a structured way of capturing context, one that the people, the records and now the agents can all draw on.

Code, then, is still the dense network I described earlier, but fewer people now trace it themselves, line by line and call by call. They interrogate it through context: an agent's summary of a module, the description on a pull request, the spec, the decision record, the

answer a model gives after reading the repository. In the terms I took from Latour, context has become the mediator between people and code, and a mediator, unlike an intermediary, transforms what passes through it. In practice, every summary of a codebase is a translation of it, and every translation can err. A team that reads its system mainly through context is reading a translation, and its understanding of the system is only as good as the context it reads through.

That is the work the CELC exists to do. If the complexity is now mirrored in context and acted upon there, the discipline that used to modularise and manage code has a new object, and the CELC is that discipline: the modularisation and management of complexity where it is now acted upon. It is the job every method on the list was doing, keeping a system within what its people can understand, carried out in context. What the discipline produces need not be an application, though. An application can be an output of a CELC, but it does not have to be. Whether it is depends on what form the data and capabilities have to take to reach the people who will use them, which may be an application or an answer rendered in somebody else's harness. Both are software in the sense I gave the word at the start.

Understandable to whom, though? I argued earlier that understandability was the objective of every method, but it is not a property a system has or lacks. It is enacted, produced each time by what a framework surfaces, in what form and through what lens. The same incident, surfaced as a timeline for the engineer on call, as an exposure for the compliance officer and as a cost for the finance lead, is three different things to understand, and none of the three is wrong. So what counts as understandable differs between organisations, because each surfaces different things to different people for its own reasons, and that is uncomfortable for anyone hoping to buy a CELC off the shelf: a finished CELC, with its stages, its records and its gates, may suit only the organisation that formed it.

What generalises is the process of forming one, and actor-network theory has a name for its first step. Callon described translation as unfolding in four overlapping moments, the first of which he called "problematisation" (Callon, 1986). In my doctoral thesis I glossed it as the moment in which "an actor identifies a problem or point of contention and formulates questions in order to both determine the identities of necessary actors, and define the problem in such a way as to establish itself as an indispensable conduit within the forming network. To do this the questions posed become obligatory passage points" (Li, 2020, p.83). The obligatory passage point I introduced at the default harness, and met again at the window and in the expert who used to interpret a business's records, is here at its origin. It is made, by whoever defines the problem first. The other three moments are how the network is then built and held: interessement, in which the actors identified are drawn in and the problematisation is tested against them; enrolment, a continual and tenuous negotiation that keeps them there; and the mobilisation of allies, in which the network, once stable enough, recruits further actors to speak for it (Callon, 1986; Li, 2020, pp.84–85).

Forming a CELC is the problematisation of AI-assisted development inside one organisation. Somebody has to define the problem so that each of the actors it involves, the engin-

eers, product owners, security, compliance, operations and the agents themselves, has to pass through the same questions to get what it wants: what context exists, where it lives, who owns it, what reaches the window, and what happens when it is wrong. The answers will differ from one organisation to the next, for the reasons I have just given. The questions, and the work of making them obligatory, are what carry over. That is the lesson of lean as I read it above: a finished form copied from another network is reassembled out of whatever the receiving network will hold, and what it will not hold is lost.

Here the historical materialist takes over again, with a sharper point: problematisation is not optional. If an organisation does not define the problem of its own adoption, somebody else already has. The default harness, as I argued at the start, is an obligatory passage point, and its vendor defined the problem in its own terms before the organisation had asked the question, so an organisation that has not asked is already enrolled in the vendor's problematisation.

Problematisation sets the questions. Something has to carry the answers between the people and agents who need them, and for that the essay needs one more term, one my own thesis never used. Susan Leigh Star and James Griesemer, studying the amateurs and professionals who together built Berkeley's Museum of Vertebrate Zoology, called such things "boundary objects": objects that "both inhabit several intersecting social worlds… *and* satisfy the informational requirements of each of them". In their definition, "Boundary objects are objects which are both plastic enough to adapt to local needs and the constraints of the several parties employing them, yet robust enough to maintain a common identity across sites. They are weakly structured in common use, and become strongly structured in individual-site use" (Star and Griesemer, 1989, p.393). A specification is the plainest case in software. The product owner reads intent in it, the engineer constraints, the auditor evidence and the agent instructions, and it holds the four together without meaning quite the same thing to any of them. The last clause of the definition is the practical one. A spec that tries to be strongly structured for everyone at once serves nobody well, which is one way to read the markdown that sank the spec-driven trials I described at the start.

Star and Griesemer used the term to qualify the model of translation I have just described. They read the account of Callon, Latour and Law as one that "can be seen as a kind of 'funnelling'… into a narrower passage point", in which "The story in this case is *necessarily* told from the point of view of one passage point – usually the manager, entrepreneur, or scientist", and they proposed in its place "a many-to-many mapping, where several obligatory points of passage are negotiated" (Star and Griesemer, 1989, p.390). They did not claim to have escaped what they named. "The analysis we propose here still contains a managerial bias," they wrote (p.390). Nor do I. A CELC formed by a CTO is told from a CTO's passage point, whatever the CTO intends.

The distinction matters to a practitioner for a further reason, which is arithmetic: what produces super-linear complexity is many-to-many arranged ad hoc. If each pair of parties builds its own passage point and its own artefacts, a handover document here and a private channel there, the links grow as $n(n-1)/2$, the same unconstrained worst

case I described for functions and for developers. Shared boundary objects are the remedy. One object that serves many interests at once replaces many pairwise ones, and the links grow roughly in step with the parties, not with their pairs. That does for context what Parnas's information hiding did for code: it keeps the number of links close to the number of parties (Parnas, 1972), though by sharing a loosely structured object rather than by hiding a decision. It is also the modularisation story the CELC needs if context is not to fragment into silos the way code fragmented into services: interests kept explicit and separate, shared boundary objects kept few, and enforced passage points kept to the fewest that will hold. What that asks of the context itself, one home for each topic and an explicit record of what supersedes what, belongs with the ways of working, and I come to it there.

Modularising context is a different question from coordinating agents, though the two cannot be cleanly separated. An actant, as I glossed it above, can only be defined by its associations, and an agent's associations run through context. It needs context to act, some of that context comes from other agents, and moving context between them takes coordination. I set out earlier the spectrum of ways to coordinate a fleet of agents, from complete control to laissez-faire, and I still do not say where on it an organisation should land. Wherever it lands, the context passing between its agents has to be coordinated, and the silo is the failure at every point on the spectrum.

A boundary object holds only for as long as somebody keeps it. Context ages. Records are superseded, descriptions drift from the code they describe, and, in the Radar's words, "obsolete context can lead AI tools astray and result in poor code generation" (Laycock et al., 2026, p.27). Nor does context age alone. When I discussed Naur I left a question open: whether the models and harnesses that now sit among a program's mediators, deprecated by their vendors or changed under the people who rely on them, are the old risk of dependency drift or a new scale of it.

Put that way, it is an older question than it looks, and it is better asked of an organisation than of the risk: who manages the model and the harness? Both readings grant that, in a program built through a model and a harness, the two touch far more of it than any one library does, but so does a cloud platform, an operating system or a database engine, and the question of who runs it has been asked of each. Where the two readings part is over who, or what, manages the model and the harness, because whoever manages them decides what can be pinned and what cannot, how much of the program stays under the organisation's control and how much depends on what a vendor does next. An organisation that builds on a vendor's ecosystem, Anthropic's or OpenAI's among many others, open-weight model families reached through aggregators included, lives with the risk at a new scale, whether or not it says so: the passage point sits with the vendor, and versions and defaults change on the vendor's schedule. An organisation that rolls its own stack, with an open-source model checkpoint and a harness it can pin, keeps it at the old scale: the passage point moves inside, and the work of keeping it moves inside with it. It is the old choice between proprietary and open source, vendor-managed and self-managed, that

organisations have made between a managed cloud and hosting their own, with a new kind of component in it.

Neither choice removes the precariousness I described when I discussed Naur; each decides only who holds it. On a vendor's stack a link can fail on a schedule the organisation does not set, and the substitute on offer may be shallow: a newer model that answers the same question differently. On its own stack the organisation holds the link, and with it the maintenance that Lehman's law says never ends. Neither is objectively worse than the other, and which suits an organisation depends on what that organisation can pin, what it must and what it can afford to keep, which is to say on its context, in the ordinary sense, and deciding that is itself work on context in mine. That is the kind of question a CELC exists to answer.

There is a limit to how much of a network can be recovered once its members have gone, and Callon gives the terms for it. No translation, he argued, is secure for good: "all translations, however apparently secure, are in principle reversible". How irreversible one becomes in practice depends on "(a) the extent to which it is subsequently impossible to go back to a point where that translation was only one amongst others; and (b) the extent to which it shapes and determines subsequent translations", and that, he added, is "a relational matter – one which can only be measured when it is put to the test" (Callon, 1991). Once a network has converged far enough, "it can be assimilated to a black box whose behaviour is known and predicted independently of its context", and the network that made it drops from view. In Callon's phrase, "The trail is cold, and the story is economised" (Callon, 1991).

When I agreed with Naur that a program's box cannot be reopened from its contents, I added "or not at any cost worth paying", and said I would come back to it. A black box becomes effectively irreversible once the cost of re-tracing its network exceeds the cost of rebuilding what it does. Nothing in principle stops the trace. Past that threshold nobody pays for it, and rebuilding is the rational course. It is the test an engineering organisation applies, usually without naming it, whenever it decides to rewrite a system instead of learning it, and it is the test Callon's "put to the test" points at.

The threshold also says where the work pays. Read this way, documentation is live, continuous tracing rather than a record made once after the fact: the network kept traceable while the trail is still warm, so that the cost of re-tracing never climbs past the cost of rebuilding. It used to be expensive, and it was the first thing dropped when a deadline came. AI assistance makes it easier than it has ever been. The same assistance that buries a repository in documentation mud when nobody manages it can keep the trace when somebody does, recording what changed, why, and which decision it answers to, as the change is made, so whether the trail goes cold is now a choice.

Every superseded record, deprecated mediator and drifting boundary object moves a black box closer to the threshold past which it is cheaper to rebuild than to understand. None of that is new in kind, only where it happens: the acting upon complexity has moved into context, where it ages, and a lifecycle built to manage code will not keep con-

text. That is the ground for the claim of succession I staked at the start, and it has two parts: the product has changed shape, into data and capabilities that agents connect, and the unit that produces it has moved, from code to context. If what has to be produced, kept, corrected and superseded is now context, the lifecycle has to be built around context, and what follows sets out what that lifecycle is and what it asks of the people working inside it.

A lifecycle for context

The Context Engineering Lifecycle is a lifecycle whose managed unit is context, where the SDLC's was functions and code. I said at the start what context is in the abstract: the medium through which computation is now directed, and a material that has to be produced, kept, corrected and superseded. Concretely, it is any recorded human communication or thought. The artefacts a project leaves behind, the user stories, feature requests and tickets, the support transcripts and emails, the decision written up after the meeting and the note somebody thought to keep: all of it is context, and any of it can reach a window. What was never recorded is not context in this sense. It may still be part of a program's theory, held by the people in the network, but no agent can act on it.

Context arrives with every degree of clarity, direction, structure and focus. A precise acceptance check sits in the same repository as a complaint forwarded through three inboxes, and a model will read both. The oldest rule in computing holds at this layer too: garbage in, garbage out. Context that is poorly organised, unstructured, unfocused or contradictory produces messy output, as badly written code always did and as badly curated training data does. So the core discipline of the CELC is the management of context. The rest of this section asks what that does to the way people work, starting with the rituals they already have.

Every method on the list I began with arrived with its ceremonies: the stand-up, sprint planning, estimation, backlog refinement, the review, the retrospective, the sign-off. Once agents write the code, it is easy to keep them all out of habit or to strip them out as overhead, and either way the question of what each one was for goes unasked. A ceremony is a payment against a cost. Each was instituted because something was expensive, and when agents implement, the thing to ask of each ceremony is what has happened to the cost it paid.

Some of those costs have fallen. Sprint planning, estimation and the triage of a defect backlog existed largely to ration implementation, which was scarce. A team batched its commitments into two weeks because two weeks was roughly what the building took, and it argued over which defects to fix because it could not fix them all. Where the building takes an afternoon, the ration is set against a scarcity that has gone, and a ceremony sized against it should shrink with it. This is the line shaft I described above, seen from the inside: the sprint is a ceremony as well as a rhythm.

Some costs have not fallen. Coordination between people costs what it always did, and probably more, because agents add actors to the network that can join it only through

what has been written down for them. The purpose of a stand-up survives that; its form need not. Alignment on intent costs more than it did, and it is paid earlier. An engineer given the wrong intent would often stall, or ask. An agent given the wrong intent produces the wrong thing, quickly and at length, so getting intent right moves upstream of the build and carries more weight there.

Verification costs more as well, in its old form. Reading what is built costs more, because more is built. Faros's 441.5% rise in median time in review (Faros, 2026) is a vendor's estimate, but its direction agrees with everything else I have cited.

Learning costs more too, and the rise is hidden. Naur held that programming is theory building, and on the reading I gave of him the theory is built across the whole network, people included. For as long as people wrote the code, their share of the theory was built as a by-product of writing it, and nobody had to schedule it. When agents write the code, the by-product stops, and whatever share people are to hold has to be built on purpose. Anthropic's trial, which I said I would return to, measured what happens when it is not: the developers who used AI assistance scored 50% against 67% for those who did not, with the largest gap on debugging (Shen and Tamkin, 2026). The pairing session, the review that taught as it checked and the retrospective all carried that purpose without naming it. Cut them for speed and the purpose goes with them, unnoticed, because nothing on a dashboard measures what a team no longer understands.

Which ceremonies survive, and in what form, will differ from one organisation to the next. They are part of the finished CELC that, as I argued above, may suit only the organisation that formed it. What one organisation can take from another is the question: what was this for, and what has happened to its price? One payment is still outstanding, for accountability, the sign-off at the end of the line. What has happened to that cost is the hardest of these questions, and it is easier to see once another is settled: how much should be specified before anything is built?

I called waterfall, in hindsight, an early form of spec-driven development, and the obvious worry about a lifecycle built on context is that it is waterfall again, made viable because executing a specification has become cheap. Some would take that as a compliment. Sean Grove puts the spec-first view as strongly as anyone: "code itself is actually a lossy projection from the specification", and so "whoever writes the spec ... is now the programmer" (Grove, 2025). The trial I cited against the bitter lesson points the other way. Colin Eberhardt, having built one app with Spec Kit, found himself "around ten times faster" without it, and concluded that "Spec Kit drags you right back into the past!" (Eberhardt, 2025). One developer and one app is thin evidence. It is still evidence, and it has the shape of waterfall's old failure: specifications that are wrong in ways only building reveals, discovered late because building was dear.

Neither side is wrong everywhere, and the most useful account I know of where each is right is Snowden and Boone's Cynefin framework. In its 2007 form it sorts the situations a leader faces into domains, among them simple (now called clear), complicated, complex and chaotic. In the complicated domain an answer can be found by analysis, though

it takes expertise to find it: "Complicated contexts may contain multiple right answers… Here, leaders must sense, analyze, and respond." In the complex domain analysis comes too early, because the answer depends on how the system and its surroundings respond, and that is known only by trying: "Leaders in this context need to probe first, then sense, and then respond." (Snowden and Boone, 2007).

Read that way, waterfall was never simply wrong. It was right in one domain and applied to all of them. Where the work is complicated, specifying first is sensing and analysing before responding, and cheap execution makes it more attractive: a good specification now turns into working code in hours. Grove is right there. Where the work is complex, nobody can analyse their way to the answer, and the specification is the lossy projection, of a system nobody has yet seen run. There Snowden and Boone's order is the right one: probe first, and write the specification after learning. Waterfall failed because the errors in a specification surfaced only when building, and building was expensive. Cheap building turns that around: the cheapest way to find out what a specification has got wrong is now, often, to build the thing and look. Where you don't know, that argues for building first; where you do, it is no argument for specifying more.

Most work is not all in one domain, and a single engagement can sit in two at once. One kind of client I have worked with across my career is an enterprise with a long-lived technical estate, whose procurement documents describe what to build but not where it must run. The what is complicated. It can be analysed, agreed and specified first. The where is complex, because what an old estate will bear is found out by connecting to it, not by reading about it. Specify the what; probe the where, with a thin slice through every integration before any feature is built on top of it.

Probing first is also how a translation is kept reversible for as long as it needs to be. Every translation, as I quoted Callon above, is in principle reversible, and what makes one irreversible in practice is how hard it becomes to return to the point where it was one option among several (Callon, 1991). A specification committed before anyone has touched the ground binds every later decision to the cheapest evidence there will ever be. A probe keeps the options open until the ground has answered, and while the trail is warm the threshold I described, past which re-tracing costs more than rebuilding, stays a long way off. Where the specification does come first, it stays honest only while it is small enough to be read in a sitting and bound to checks that fail when the code drifts from it. A specification nobody can read is documentation mud with a plan attached.

Whichever comes first, the specification or the probe, something gets built, and less and less of it is typed by a person. That returns me to the payment I set aside, for accountability, and the answer about specification helps: if what is specified first depends on the domain, so does what can be checked first, and the checking is where accountability now sits.

There are two easy answers. One keeps accountability where it was, with a person who reads the code, and lets verification grow until it keeps up. The other says that if generation is a given, verification can be generated too, and the person signing off can go the

way of the person typing. They look opposed, and they share an assumption: that verification happens at the code layer. The first adds people to read more code. The second has a machine read it instead. Either way verification stays belted to the old line shaft. One answer adds hands to turn it, and the other adds a motor.

I think verification has to come off the code layer altogether. If the complexity of code has been abstracted into context, the complexity of verifying it should be abstracted the same way, and what we review now is context, not code.

It helps to remember why code review was there. In the SDLC, before coding models, review was an obligatory passage point that the economics of the SDLC made: the first, default and cheapest way to get some control over quality, and in most teams no change reached the codebase without passing through it. Nobody chose it as a matter of opinion. It was simply cheaper than the alternative. Building a test suite cost more than having a colleague read a diff, especially for a minor functional change, and executing a program end to end, with a test environment set up, a full compile and a deployment, cost development cycles that were spent only on the functions that could not be allowed to fail. So review was where checking happened, and full testing happened only where it mattered most.

That cost differential has collapsed: the cost of verifying by execution has fallen as the cost of verifying by reading has risen. The argument here is my own, because I know of no study of the costs that I would trust. What I can point to is scale. Engineers at Meta have described generated tests improving existing, human-written suites in the company's test-a-thons (Alshahwan et al., 2024), and a second system that was "applied to 10,795 Android Kotlin classes in 7 software platforms", where "engineers accepted 73% of its tests" (Foster et al., 2025). Neither paper reports what the tests cost to produce or to run, so neither proves my point. They show that generated tests have left the laboratory. Generating a suite is now cheap. Running one is not free, and I do not claim it is, but it is much cheaper than it was, and an agent can now take a program through full execution as a matter of routine, where a team once kept that for the few functions it could not afford to get wrong.

What follows for code review is that it has stopped being obligatory. With AI assistance it is a choice, and, like choosing how much to automate, an opinion, so somebody has to hold it consciously: the technology leaders and whoever else sets the standards. Held consciously, it becomes two other decisions. One is about learning and development. Review was one of the ways engineers learned a codebase and one another, and an organisation that still has to grow the people who will succeed its senior engineers may keep human review for that reason, which is the hidden cost of learning I described above. The other is about compliance. Where review was there to keep standards, telemetry, governance and randomised tracing can keep them instead, the way any organisation oversees contractors whose work it cannot watch: it instruments the work, sets rules for it and audits samples of it. Both are decisions about the records, rules and telemetry around the code, which is to say about context, and so they belong to the CELC.

If checking leaves the code, it has to land somewhere, and where it lands is execution against criteria. What counts is whether it works as it should, which is more than whether it runs: at scale, under pressure, while failing, while being attacked, within its budget, and in a way, and with an experience, acceptable to the people who will use it day to day. Those are the criteria of a program that is well architected, and they can be tested now that testing is cheap. What an agent produces against them is not proof in any sense a logician would accept, since running a program shows only what it did on the runs that were made, and when the code was generated by a process that would not produce it the same way twice, the runs are all there is, which is the sideways step I described at the start. It is demonstration and evidence. The demonstration can be kept as an artefact, a recording of the program doing what the criteria asked of it. The evidence is the reasoning trace, the logs and the telemetry of the run.

Which leaves the criteria, and here the person stays: the criteria are the governed portion I set aside in answering the bitter lesson, small in proportion, not in importance, and drafted with a model's help even where a person decides. A model asked to write the tests for its own work will tend to write tests it knows it can pass. A related behaviour, gaming the tests it has been given, has been measured. ImpossibleBench sets coding agents tasks whose tests contradict their specifications, so that any pass is a shortcut, and its authors state the problem in a line: "an LLM agent with access to unit tests may delete failing tests rather than fix the underlying bug" (Zhong et al., 2025). So a person defines correct execution, not execution in general, because execution in general invites exactly that kind of reward hacking, gaming the measure. Handing the criteria to a second model does not solve it. A model graded by another model cannot be graded higher than the grader can recognise, and where the grader is a frontier model, as it is wherever models are set to grade models, that model sets the ceiling. The bottleneck moves to the author of the criteria. I accept that. It is the right place for it, because the author of the criteria is somebody who can be held to account.

Every step of a CELC may well be automatable, but some are better augmented than automated, because what they preserve is human judgement and an opportunity for alignment, and a lifecycle should say which steps those are instead of drifting into automating them. Authoring the criteria is one. Its author can be helped by a model at every turn, drafting, enumerating edge cases, looking for gaps, and still be the one who decides what counts. The gate at the back of the build is the other, where the evidence is set against the criteria, because it is the last opportunity to exercise judgement and to align, or if need be to realign, before the work reaches the people it was built for.

The gate at the back connects to the gate at the front. Gathering context before generation is, as I argued earlier, the first gate, and it is also where the foundations of the tests are laid. In that sense a CELC takes elements of both spec-driven and test-driven development, because AI has taken over the grunt work of implementing each. It used to be prohibitively expensive, in cost and in velocity, to go both ways, so a team either front-loaded its verification into a specification or back-loaded it into tests. A CELC can do some of each without adopting either wholesale. Having argued that the depth of a spe-

cification should be set by the domain, I could hardly then adopt one method's answer for every domain.

Putting criteria at the centre raises two further questions. The first is whether the builder should see the criteria it will be checked against. It should. An agent building against criteria it cannot see is guessing, and the criteria are among the context it needs. The exception is a separate evaluation suite, run by fresh agents, where the validation criteria are sealed away and checked by an agent other than the one that tested the work. That is also where independence comes from. The demand for an independent check is old, and meeting it does not need a human reader, only a checker with no part in the thing it checks.

The second question is whether criteria can be fixed in advance in work that, as I argued above, should often be probed first. They need not be. Criteria are context like any other, and they have a lifecycle of their own: provisional at the start, hardening progressively as the work approaches delivery, much as development always hardened in the SDLC. The probe is often how they harden. Criteria also creep, as scope does, and for the same reason, since a criterion is what scope looks like once it can be tested. Managing the creep falls to the author of the criteria, and how it gets managed depends, as everything in an actor-network does, on a great many other things: timing, the people and money available, licensing and procurement, contracts under negotiation, the business cycle, a vendor changing its terms. Nor is the gate at the back the end of anything. Software is not finished when it reaches production. Somebody still has to say that this is good enough for the people it was built for, and the lifecycle carries on past that point.

Somebody, then, is accountable. Accountability has not been absorbed into the agent because the layer being checked has moved. It has moved up the stack with it, and a person is still responsible. It rests with a person, but not because people read code better than models; for most engineers that stopped being true a while ago, and it becomes less true with every model released. The reason is imputation: only a person can be held to account. Imputation still has a threshold: whether a person can follow the program into its substrate when it breaks. Reading code better than a model is no longer part of it. Processes can be improved and stress-tested until they are reliable, and those processes are actants in their own right, but the buck has to stop somewhere, and in the world as it is the norms of our culture put a person there, whether one likes it or not.

That might look like the point at which actor-network theory has to be set aside, since a theory that treats people and things symmetrically seems to have no reason to single out a person to answer for a failure. It does not have to be set aside. Accountability does not have to leave the network to land on a person. Callon's study followed three researchers trying to restock scallops in St Brieuc Bay, in Brittany, by getting larvae to anchor on collectors, with the fishermen's agreement. In St Brieuc Bay three researchers came to speak for the scallops, and the fishermen were spoken for by their elected representatives. To translate, in Callon's account, is "to express in one's own language what others say and want, why they act in the way they do and how they associate with each other: it is to establish oneself as a spokesman" (Callon, 1986). The position is precarious. In the years

after the first experiment the collectors stayed empty, and one Christmas Eve the fishermen fished out the scallops that had been gathered in a protected part of the bay. The spokespeople were shown to have spoken for nobody. Any such consensus, Callon warned, "can be contested at any moment. Translation becomes treason" (Callon, 1986).

In an actor-network, then, being held to account is a position a person is made to occupy, and can be stripped of when the represented refuse the part written for them. In a context lifecycle the author of the criteria holds that position. They speak for the end user, the budget, the attacker who has not yet arrived and the agents who will build against what they write. When the build betrays the criteria, it is their representation that has failed, and they who must trace why. That a person holds it is, as I said, a matter of culture; actor-network theory does not need to endorse that in order to describe it, and it explains why the position holds only while the person can still trace what they speak for.

Tracing is where expertise in the code itself still counts. Translations break down, and in the day-to-day building of systems they invariably do. When one does, somebody has to open the box, follow the program into its substrate and find where it came apart. That is an engineer's troubleshooting, and it is, step for step, what an actor-network theorist does in tracing a network. Only the setting differs.

It also divides the people now able to build software. A model and a harness can interpret the inscrutability of code for anyone, so somebody who never understood code can now build with it. A machine translator does something similar with a phrase in Mandarin: it gives an English speaker the surface meaning, and not the subtext or the culture behind it. A coding model gives a lay person about as much. An engineer who understood code before taking up the same tools is in a different position. To be accountable for a program is to be able to de-black-box what a model has produced in it. Somebody who cannot is a spokesperson who does not speak the language of those they represent, in the position of a person who signs a contract they cannot read. Call it negligence, or something worse. The person in that position falls below the threshold imputation sets: they are not capable of assuming the accountability, and arguably should never have been put there, but the best-laid plans of mice and men often go awry. The best a naive vibe coder can do with a black box is divide it into smaller black boxes.

The criteria are context, and so is everything else in this section, which brings me to the duty I deferred when I described boundary objects: one home for each topic, and an explicit record of what supersedes what. Context ages, as I said then, and it does not all age the same way. Records, the decisions taken, the experiments run and the incidents written up, do not go stale; what happened, happened. Their status changes, though, when something supersedes them, and a superseded decision left where an agent can read it still looks like a decision. Descriptions, the specifications, rules and reference documents that say how things are, drift away from what they describe as it changes underneath them. Transients, the tickets, the feedback and an agent's memory of its last session, are meant to be used and let go, and they should expire. Treat all three as documentation and each fails in its own way. Agents make every one of those failures dearer, because an agent reads whatever it is given and treats it as current.

Contradiction is the worst of these failures, and review cannot be relied on to catch it. Two pieces of context that disagree will each be acted on by something, and the model in the middle has no reliable way to tell which was meant. So contradiction has to be prevented by structure, not by effort: each topic has one home, and when something supersedes something else, the record says so. It is the modularisation I argued for with boundary objects, applied over time as well as across the network. It is also how the learning loop I took from Toyota reaches agents. A setback written up in a retrospective changes nothing an agent does. A setback traced to its cause, and fed into the criteria, the rules or the specification that allowed it, changes what every agent does next.

The same structure is what lets the CELC keep the choice I made when I described the continuum. I said it lands at the quality end, treating the cognitive load of oversight as something to manage, not something to absorb. Everything in this section moves that load. Verification leaves the code and lands on the criteria and the gate, and the people at the gate carry the judgement whose load Bainbridge foresaw and the brain-fry survey was a first attempt to measure (Bainbridge, 1983; Bedard et al., 2026). Managing it needs no new idea. A limit on how much work may sit in review and verification at once, a limit on work in progress, already manages it. Past the limit nothing new starts until something waiting has been judged, so the queue shows up as a queue, where everyone can see it, instead of as fatigue in the people at the end of the line. It is Toyota's cord, moved from the assembly line to the review column, and harder to pull for the reason I gave: more is in motion when it stops. Pulling it is a choice about the relations of production, about who carries the load of oversight and on what terms: cost-cutting absorbs the load into whoever is left, and the CELC manages it.

That is the lifecycle, as far as an essay should take it; what it looks like in any one organisation is for that organisation to form.

The wall at the edge of the firm

One question remains before the close, because a lifecycle that lets software be shaped to how an organisation works has to say how far that shaping can go. Sloan's lesson, as I read it earlier, was that variety could be planned, produced and paid for at scale, and AI-assisted development reached its Sloanist stage almost at once. The same cheapness, taken a step further, makes software customisable to a degree it has never been. For as long as engineering cycles were scarce, organisations faced a hard choice. They could buy generic software off the shelf and bend the way their people worked to fit it, or they could spend heavily on internal tools of their own, which tended to become technical debt that nobody could afford to pay down. When generation is cheap, that choice softens. Technically, at least, software can now be shaped to how a particular organisation, or a team inside one, actually works.

The obvious place to look for a limit to that is Conway's law, which I quoted when I described how the shape of an organisation gets into its code. Read one way, cheap generation makes the law cheap to obey: build software that copies how the organisation actually communicates, and rebuild it when that changes. Read the other way, the law still

binds, and what it copies now includes the agents and whatever they have been given to read. Both readings treat the organisation as one thing and its software as another, as though every unit bound to its software in the same way and used it the same way. Dissected, that is preposterous. Organisations are heterogeneous, with functions that differ in what they do, what they can do and who does it, and so is software, if only because its vendors work so hard to tell their products apart. So whether Conway's law is cheap to obey is too blunt a question; the answer is graduated, and a lot messier.

Start with what holds an organisation together at all. It is aligned at least to the extent of sharing some goals, or it would be a list of actants with no connection to one another and no influence on one another. That alignment has to be achieved, and it is achieved to different degrees in different places, which is how a business unit can fail in its objective while the wider business succeeds. In the terms I took from Callon, it is enrolment, and enrolment is never finished. Towards the core of an organisation more of its body is enrolled, and its systems tend to be more homogeneous, because that is how the benefits of scale are captured and how relationships are drawn laterally across it. Towards the edge the systems can grow heterogeneous, all the way down to a single user or operator with a personal dashboard and a knowledge base of their own, who can still enrol a dataset or a capability shared by a much wider unit. A young company shows the core forming. Its founders start out holding several roles each, and as they hand the excess off, what lived in one or two heads has to move into the boundary objects between the roles.

Where the core ends and the edge begins depends on how the units are drawn, and the organisation chart is no guide. It is a spokesperson, and an arbitrary one. Organisations break into units and sub-units, teams and shifts, cliques and movements, formal groups and informal ones, cutting across functional, cultural, social and economic lines. The units nest, like a matryoshka doll, and they overlap, since one actant can belong to two groupings at once and every grouping has groupings inside it. So, depending on the context, a system can obey Conway's law and not obey it at the same time: it mirrors the unit it serves at one level and cuts across units at another. The two states are not in tension, because the organisational setting divides them, and where each holds differs from one organisation, and one unit, to the next.

None of this is exotic. The internet was built on what Jerome Saltzer, David Reed and David Clark named the end-to-end argument: a function that "can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system" belongs at the ends (Saltzer et al., 1984). Put the function at the ends, and the network in the middle can stay simple and common to everyone, while the variety lives at the edges. A federation has the same shape, with national law, states legislating within it and councils making their by-laws, and a citizen subject to all three at once; so does a standard language, with a shared grammar at its core and dialects, jargon and idiolects at its edges, several of which everyone speaks. None of them is either centralised or decentralised: each has a uniform core and heterogeneous edges, and its members live in both.

Here, as everywhere else, AI adds the capacity to manage a great deal more complexity in any one context. Retrieval-augmented generation is the plain case, a model mediating between a person and far more material than the person could hold. With that mediation the uniform core can grow larger and more complex, and so it moves outward, and Conway's law becomes cheaper to obey for larger and less uniform units. It does not move for free. A company may grow linearly, adding people and teams at a steady rate, but the relationships between the people, teams and systems inside it grow superlinearly, as I argued for code: a codebase can add lines linearly while the relations between its units multiply, and Brooks's $n(n-1)/2$ is the same arithmetic for people (Brooks, 1975), which a widening core inherits.

So there are limits, and the quickest way to find them is to take the stance too literally, shoving in more data and building more functions because a model will mediate them. Models do not use what they are given evenly. Nelson Liu and his colleagues found performance "often highest when relevant information occurs at the beginning or end of the input context", and degrading significantly when models "must access relevant information in the middle of long contexts, even for explicitly long-context models" (Liu et al., 2023). Chroma, a database vendor, tested 18 models in 2025 and reported that "their performance grows increasingly unreliable as input length grows" (Hong et al., 2025). A core widened by naively stuffing it is context left unmanaged, and the second law again applies: its complexity grows unless work is done to hold it back. The complexity I argued migrates into context does not stop migrating because the window got longer, and an organisation that takes the stance literally invites a Cambrian explosion of it. Exactly where the limits fall I cannot say for everyone; each organisation has to find its own.

Somebody has to decide where each unit sits on that scale, between the uniform core and the personal edge. A leader is one example of an organisational spokesperson, in the sense I gave the word when I discussed accountability, and deciding that is a spokesperson's work: at each level of the organisation, or through the leaders delegated to each level of a large one, judging how far a unit should align to Conway's law, given its size and how uniform it is. The same spokesperson has a second job now, which is to manage how fast complexity is allowed to scale and how it is pruned, so that the organisation can hold what it builds.

Every unit I have described sits inside larger ones, and the nesting has to stop somewhere if the argument is to say anything. I stop it at the wall, the boundary of the firm, and that is a choice, but not an arbitrary one: that boundary is where the shared goals stop for that organisational unit as well, and shared goals are what align a unit at all. The firm also remains society's basic unit of economic organisation, and Ronald Coase gave the reason firms exist at all: directing production inside a firm can cost less than using the price mechanism (Coase, 1937). To make a profit or capture a rent, a firm has to keep some scale, or its production fragments into a cottage industry nobody can manage. The wall is porous, though: the harness crosses it, since the interface a firm's people see may be set by a vendor outside the firm, which is why problematising one's own adoption matters. So does almost every dependency that is not worth the firm's while to make it-

self. Cheap generation moves Coase's boundary, since building and changing software inside the firm now costs less than it did, but it does not abolish the firm. My working claim, then, is that customisation will track the units of self-organisation inside the firm, up to the firm's edge, and that AI allows a much closer alignment between software and how a firm actually organises itself than the SDLC ever did. Much closer is as far as I would put it.

The reorganisation lag

I end where the history left off, with the dynamo. David's factories waited four decades for their gains, and the reason was material: the old plant had to wear out before replacing it paid (David, 1990, p.357). I have argued that this transition lags for a different reason, a shortage not of capital but of imagination: the capacity of organisations formed around one set of tools to picture themselves rebuilt around another, and its shortage holds up a reorganisation that runs from ways of working through the infrastructure of society, business, government, media and technology. The material limits are real, but on a longer horizon than today's.

That shortage is the organising problem, and everything above bears on it. On David's evidence the gains came with the rebuild, not the motor, and I expect the gains from agents to go, here too, not to everyone who uses them but to those who rebuild how they work around context: who treat context as the unit that is produced, kept, corrected and superseded, who move verification off the code and onto criteria somebody answers for, and who decide, unit by unit, how closely their software should follow the way each unit actually works. Running agents inside the old line shaft buys a new motor and keeps the old factory. The code arrives faster and piles up at review, as the evidence I began with already shows, and the gain stays out of sight.

An essay cannot say how. This one has argued that software is being redefined, that the lifecycle built for the old thing has to be succeeded, not supplemented, and that the complexity is now mirrored in context, where it is acted upon and where it ages. How any one organisation rebuilds around that is a question of method, and it needs a different kind of document, a follow-up whitepaper. What generalises, as I argued, is not a finished CELC but the process of forming one, and that is what a method can describe. I can at least state the question: what does an organisation have to change, and in what order, to rebuild how it works around context instead of running agents on the old shaft? Until that is answered, the dynamos will keep turning, and the productivity statistics will keep missing them.

References

Alshahwan, N., Chheda, J., Finegenova, A., Gokkaya, B., Harman, M., Harper, I., Marginean, A., Sengupta, S. and Wang, E. (2024) "Automated Unit Test Improvement using Large Language Models at Meta", *FSE 2024* (industry track), arXiv:2402.09171, 14 February, https://arxiv.org/abs/2402.09171.

Anthropic (2025) "Anthropic Economic Index: AI's Impact on Software Development", Anthropic, 28 April, https://www.anthropic.com/research/impact-software-development.

Arcolano, N. (2025) "What Data from 20m Pull Requests Reveal About AI Transformation", talk at AI Engineer Code Summit, New York, November, video, https://www.youtube.com/watch?v=WqZq8L-v9pA.

AWS (2025) "AI-Driven Development Life Cycle: Reimagining Software Engineering", by Raja SP, *AWS DevOps Blog*, 31 July, https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/.

Bainbridge, L. (1983) "Ironies of Automation", *Automatica*, 19(6), pp.775–779.

BCG (2025) *Executive Perspectives: AI-Enabled Engineering Excellence Transformation*, Boston Consulting Group, April, https://www.bcg.com/assets/2025/executive-perspectives-ai-enabled-engineering-excellence-23april.pdf.

Bedard, J., Kropp, M., Hsu, M., Karaman, O.T., Hawes, J. and Kellerman, G.R. (2026) "When Using AI Leads to 'Brain Fry'", *Harvard Business Review*, 5 March, https://hbr.org/2026/03/when-using-ai-leads-to-brain-fry.

Benioff, M. (2025) Remarks in Salesforce, Inc., FQ4 2025 Earnings Call Transcript, 26 February, S&P Global Market Intelligence, https://s205.q4cdn.com/626266368/files/doc_financials/2025/q4/Salesforce-Inc-_Earnings-Call_2025-02-26_English.pdf.

Bick, A., Blandin, A. and Deming, D.J. (2024) "The Rapid Adoption of Generative AI", NBER Working Paper 32966, National Bureau of Economic Research, September, revised February 2025, doi:10.3386/w32966, https://www.nber.org/papers/w32966.

Böckeler, B. (2025) "Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl", *martinfowler.com*, 15 October, https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html.

Braverman, H. (1974) *Labor and Monopoly Capital: The Degradation of Work in the Twentieth Century*, Monthly Review Press, New York.

Brooks, F.P. (1975) *The Mythical Man-Month: Essays on Software Engineering*, Addison-Wesley, Reading, MA.

Brooks, F.P. (1987) "No Silver Bullet: Essence and Accidents of Software Engineering", *Computer*, 20(4), pp.10–19, doi:10.1109/MC.1987.1663532.

Brynjolfsson, E., Chandar, B. and Chen, R. (2026a) "Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of Artificial Intelligence", Stanford Digital Economy Lab, August revision, https://digitaleconomy.stanford.edu/app/uploads/2026/08/Canaries_August2026.pdf.

Brynjolfsson, E., Chandar, B. and Chen, R. (2026b) "Canaries, Interest Rates, and Timing: More on the Recent Drivers of Employment Changes for Young Workers", Stanford Digital Economy Lab, 9 February, https://digitaleconomy.stanford.edu/news/canaries-interest-rates-and-timinga-more-on-recent-drivers-of-employment-changes-for-young-workers/.

Callon, M. (1986) "Some Elements of a Sociology of Translation: Domestication of the Scallops and the Fishermen of St Brieuc Bay", in Law, J. (ed.) *Power, Action and Belief: A New Sociology of Knowledge?*, Sociological Review Monograph 32, Routledge & Kegan Paul, London, pp.196–233.

Callon, M. (1991) "Techno-economic Networks and Irreversibility", in Law, J. (ed.) *A Sociology of Monsters: Essays on Power, Technology and Domination*, Routledge, London, pp.132–161.

Callon, M. and Latour, B. (1981) "Unscrewing the Big Leviathan: How Actors Macro-Structure Reality and How Sociologists Help Them to Do So", in Knorr-Cetina, K. and Cicourel, A.V. (eds) *Advances in Social Theory and Methodology: Toward an Integration of Micro- and Macro-Sociologies*, Routledge & Kegan Paul, Boston and London, pp.277–303.

Capoot, A. (2026) "Anthropic IPO meets AI backlash after midterms show stinging rebuke of data centers", *CNBC*, 7 November, https://www.cnbc.com/2026/11/07/anthropic-ipo-ai-data-center-backlash.html.

Coase, R.H. (1937) "The Nature of the Firm", *Economica*, New Series, 4(16), pp.386–405, doi:10.1111/j.1468-0335.1937.tb00002.x.

Comin, D. and Hobijn, B. (2010) "An Exploration of Technology Diffusion", *American Economic Review*, 100(5), pp.2031–2059, doi:10.1257/aer.100.5.2031.

Conway, M.E. (1968) "How Do Committees Invent?", *Datamation*, 14(4), pp.28–31.

David, P.A. (1990) "The Dynamo and the Computer: An Historical Perspective on the Modern Productivity Paradox", *American Economic Review*, 80(2), Papers and Proceedings, pp.355–361.

Debois, P. (2026) "Context Development Lifecycle", *Tessl Blog*, 19 February, https://tessl.io/blog/context-development-lifecycle-better-context-for-ai-coding-agents.

Delimarsky, D. (2025) "Spec-driven development with AI: get started with a new open source toolkit", *The GitHub Blog*, 2 September, https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/.

DORA (2024) "Announcing the 2024 DORA report", *Google Cloud Blog*, 23 October, https://cloud.google.com/blog/products/devops-sre/announcing-the-2024-dora-report.

Eberhardt, C. (2025) "Putting Spec Kit Through Its Paces: Radical Idea or Reinvented Waterfall?", *Scott Logic Blog*, 26 November, https://blog.scottlogic.com/2025/11/26/putting-spec-kit-through-its-paces-radical-idea-or-reinvented-waterfall.html.

Faros (2026) "AI acceleration whiplash", Faros AI Blog, 12 April, https://faros.ai/blog/ai-acceleration-whiplash-takeaways.

Ford, H. and Crowther, S. (1922) *My Life and Work*, Doubleday, Page & Company, Garden City, NY, Project Gutenberg eBook #7213, https://www.gutenberg.org/ebooks/7213.

Fore, P. (2026) "IBM is tripling the number of Gen Z entry-level jobs after finding the limits of AI adoption", *Fortune*, 13 February, https://fortune.com/2026/02/13/tech-giant-ibm-tripling-gen-z-entry-level-hiring-according-to-chro-rewriting-jobs-ai-era/.

Foster, C., Gulati, A., Harman, M., Harper, I., Mao, K., Ritchey, J., Robert, H. and Sengupta, S. (2025) "Mutation-Guided LLM-based Test Generation at Meta", arXiv:2501.12862, 22 January, https://arxiv.org/abs/2501.12862.

Fowler, M. (2025) "LLMs bring new nature of abstraction", *martinfowler.com*, 24 June, https://martinfowler.com/articles/2025-nature-abstraction.html.

GitClear (2025) "AI Copilot Code Quality: 2025 Look Back at 12 Months of Data", GitClear, https://www.gitclear.com/ai_assistant_code_quality_2025_research.

Grove, S. (2025) "The New Code", talk at AI Engineer World's Fair, San Francisco, 3–5 June, video posted 11 July, https://www.youtube.com/watch?v=8rABwKRsec4.

Harvey, N. and DeBellis, D. (2025) "Announcing the 2025 DORA Report", *Google Cloud Blog*, September, https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report.

Hassan, A.E., Oliva, G.A., Lin, D., Chen, B. and Jiang, Z.M. (2024) "Towards AI-Native Software Engineering (SE 3.0): A Vision and a Challenge Roadmap", arXiv:2410.06107, 8 October, https://arxiv.org/abs/2410.06107.

Hong, K., Troynikov, A. and Huber, J. (2025) "Context Rot: How Increasing Input Tokens Impacts LLM Performance", Chroma Technical Report, 14 July, https://www.trychroma.com/research/context-rot.

Jassy, A. (2024) Post on LinkedIn, 22 August, https://www.linkedin.com/posts/andy-jassy-8b1615_one-of-the-most-tedious-but-critical-tasks-activity-7232374162185461760-AdSz/.

Jassy, A. (2025) "Message from CEO Andy Jassy: Some thoughts on Generative AI", *About Amazon*, 17 June, https://www.aboutamazon.com/news/company-news/amazon-ceo-andy-jassy-on-generative-ai.

Johnson, E.J. and Goldstein, D. (2003) "Do Defaults Save Lives?", *Science*, 302(5649), pp.1338–1339, doi:10.1126/science.1091721.

Karpathy, A. (2017) "Software 2.0", *Medium*, 11 November, https://karpathy.medium.com/software-2-0-a64152b37c35.

Karpathy, A. (2025a) "Software Is Changing (Again)", keynote address, YC AI Startup School, 17 June, https://www.youtube.com/watch?v=LCEmiRjPEtQ.

Karpathy, A. (2025b) Post on X, 2 February, https://x.com/karpathy/status/1886192184808149383.

Karpathy, A. (2025c) Post on X, 25 June, https://x.com/karpathy/status/1937902205765607626.

Karpathy, A. (2026) Post on X, 4 February, https://x.com/karpathy/status/2019137879310836075.

Kim, Y. and Liu, X. (2026) "Towards a science of scaling agent systems: When and why agent systems work", *Google Research Blog*, 28 January, https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/.

Koc, V. (2026) "Dark Factory: OpenClaw ships faster than you can read the diff", talk at AI Engineer Europe 2026, London, 8–10 April, video posted 5 June, https://www.youtube.com/watch?v=pmoDeA3RBZY.

Latour, B. (1996) "On Actor-Network Theory: A Few Clarifications", *Soziale Welt*, 47(4), pp.369–381.

Latour, B. (1999) *Pandora's Hope: Essays on the Reality of Science Studies*, Harvard University Press, Cambridge, MA.

Latour, B. (2005) *Reassembling the Social: An Introduction to Actor-Network-Theory*, Oxford University Press, Oxford.

Law, J. (1992) "Notes on the Theory of the Actor-Network: Ordering, Strategy, and Heterogeneity", *Systems Practice*, 5(4), pp.379–393, doi:10.1007/BF01059830.

Laycock, R. et al. (2025) *Technology Radar*, Vol. 33, November, Thoughtworks, https://www.thoughtworks.com/radar/techniques/spec-driven-development.

Laycock, R. et al. (2026) *Technology Radar*, Vol. 35, November, Thoughtworks, https://www.thoughtworks.com/radar/tools/openspec.

Lehman, M.M. (1980) "Programs, Life Cycles, and Laws of Software Evolution", *Proceedings of the IEEE*, 68(9), pp.1060–1076.

Li, R. (2011) *'Killing the Noob': A Thesis on Meritocracy and What It Has to Do with Massive Multiplayer Online Role Playing Games*, Bachelor of Arts (Honours) (Media, Games and Multimedia) thesis, Swinburne University of Technology.

Li, R. (2020) *Reconceptualizing MMORPG Play: A Study on Actor-Network Theory in Practice*, PhD thesis, Swinburne University of Technology.

Li, R. (2025) "The Evolution of Human-Internet Interaction: Toward a New Information Architecture", *drli.blog*, 9 June, https://drli.blog/posts/agentic-web-evolution/.

Li, X., Wang, A., Wang, G., Li, J. and Shum, C. (2025) "CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning", arXiv:2507.14111, 18 July, https://arxiv.org/abs/2507.14111.

Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F. and Liang, P. (2023) "Lost in the Middle: How Language Models Use Long Contexts", arXiv:2307.03172, 6 July, https://arxiv.org/abs/2307.03172.

Lopopolo, R. (2026) "Harness engineering: leveraging Codex in an agent-first world", OpenAI, 11 February, https://openai.com/index/harness-engineering/.

Lukács, G. (1971) "Reification and the Consciousness of the Proletariat", in *History and Class Consciousness: Studies in Marxist Dialectics*, trans. R. Livingstone, Merlin Press, London (first published 1923).

Lütke, T. (2025) Post on X, 19 June, https://x.com/tobi/status/1935533422589399127.

MacCormack, A., Baldwin, C. and Rusnak, J. (2012) "Exploring the Duality between Product and Organizational Architectures: A Test of the 'Mirroring' Hypothesis", *Research Policy*, 41(8), pp.1309–1324.

METR (2025) "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity", METR, 10 July, https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/.

METR (2026) "We are Changing our Developer Productivity Experiment Design", METR, 24 February, https://metr.org/blog/2026-02-24-uplift-update/.

Nagappan, N., Murphy, B. and Basili, V. (2008) "The Influence of Organizational Structure on Software Quality: An Empirical Case Study", in *Proceedings of the 30th International Conference on Software Engineering (ICSE '08)*, ACM, New York, pp.521–530.

Naur, P. (1985) "Programming as Theory Building", *Microprocessing and Microprogramming*, 15(5), pp.253–261, doi:10.1016/0165-6074(85)90032-8.

Ohno, T. (1988) *Toyota Production System: Beyond Large-Scale Production*, Productivity Press, Cambridge, MA (first published in Japanese 1978).

Ouyang, A., Guo, S., Arora, S., Zhang, A.L., Hu, W., Ré, C. and Mirhoseini, A. (2025) "KernelBench: Can LLMs Write Efficient GPU Kernels?", arXiv:2502.10517, 14 February, https://arxiv.org/abs/2502.10517.

Parker, M. and Slaughter, J. (1988) *Choosing Sides: Unions and the Team Concept*, South End Press, Boston.

Parker, M. and Slaughter, J. (1989) "Management's 'Ideal' Concept", *Against the Current*, no. 19, March/April, https://againstthecurrent.org/atc019/managements-ideal-concept/.

Parnas, D.L. (1972) "On the Criteria To Be Used in Decomposing Systems into Modules", *Communications of the ACM*, 15(12), pp.1053–1058.

Peng, S., Kalliamvakou, E., Cihon, P. and Demirer, M. (2023) "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot", arXiv:2302.06590, 13 February, https://arxiv.org/abs/2302.06590.

Reuters (2026) "Global AI debt issuance to top $500 billion in 2026, Morgan Stanley says", Reuters, 10 June, https://finance.yahoo.com/sectors/technology/articles/global-ai-debt-issuance-top-085023868.html.

Saltzer, J.H., Reed, D.P. and Clark, D.D. (1984) "End-to-End Arguments in System Design", *ACM Transactions on Computer Systems*, 2(4), pp.277–288.

Shen, J.H. and Tamkin, A. (2026) "How AI assistance impacts the formation of coding skills", Anthropic, 29 January, https://www.anthropic.com/research/AI-assistance-coding-skills.

SignalFire (2025) "The SignalFire State of Tech Talent Report – 2025", *SignalFire Blog*, 20 May, https://www.signalfire.com/blog/signalfire-state-of-talent-report-2025.

Slesarenko, A. (2025) "Context Engineering 101: Architecting Smarter Multi-Agent Systems", *OneReach.ai Blog*, 8 September, https://onereach.ai/blog/smarter-context-engineering-multi-agent-systems/.

Snowden, D.J. and Boone, M.E. (2007) "A Leader's Framework for Decision Making", *Harvard Business Review*, 85(11), pp.68–76, 149.

Solow, R.M. (1987) "We'd Better Watch Out", *New York Times Book Review*, 12 July, p.36.

Star, S.L. and Griesemer, J.R. (1989) "Institutional Ecology, 'Translations' and Boundary Objects: Amateurs and Professionals in Berkeley's Museum of Vertebrate Zoology, 1907–39", *Social Studies of Science*, 19(3), pp.387–420, doi:10.1177/030631289019003001.

Sutton, R. (2019) "The Bitter Lesson", *Incomplete Ideas*, 13 March, http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

Swaminathan, N. and Singh, D. (2025) "Introducing Kiro", *Kiro Blog*, 14 July, https://kiro.dev/blog/introducing-kiro/.

Tane, B. (2026) "The Software Development Lifecycle Is Dead", 21 February, https://boristane.com/blog/the-software-development-lifecycle-is-dead/.

Tedlow, R.S. (1988) "The Struggle for Dominance in the Automobile Market: The Early Years of Ford and General Motors", *Business and Economic History*, 2nd series, 17, pp.49–62.

The Henry Ford (n.d.a) "Henry Ford and Edsel Ford in Fifteen-Millionth Ford Model T Car on the Last Day of Model T Production, May 26, 1927", Digital Collections, artifact 280343, https://www.thehenryford.org/collections/explore/artifact/280343.

The Henry Ford (n.d.b) "Edsel Ford and Henry Ford Introducing the New Model A at the Ford Industrial Exposition in New York City, January 1928", Digital Collections, artifact 176600, https://www.thehenryford.org/collections/explore/artifact/176600.

United States v Google LLC (2024) Memorandum Opinion, Case No. 20-cv-3010 (APM), United States District Court for the District of Columbia, ECF No. 1033, 5 August, https://storage.courtlistener.com/recap/gov.uscourts.dcd.223205/gov.uscourts.dcd.223205.1033.0_2.pdf.

United States v Google LLC (2025) Memorandum Opinion (remedies), Case No. 20-cv-3010 (APM), United States District Court for the District of Columbia, ECF No. 1436, 2 September, https://storage.courtlistener.com/recap/gov.uscourts.dcd.223205/gov.uscourts.dcd.223205.1436.0.pdf.

Varas, A., Varadarajan, R., Goodrich, J. and Yinug, F. (2021) *Strengthening the Global Semiconductor Supply Chain in an Uncertain Era*, Boston Consulting Group and Semiconductor Industry Association, April.

Womack, J.P., Jones, D.T. and Roos, D. (1990) *The Machine That Changed the World*, Rawson Associates, New York.

Zhong, Z., Raghunathan, A. and Carlini, N. (2025) "ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases", arXiv:2510.20270, 23 October, https://arxiv.org/abs/2510.20270.